Inatel

Tópicos Avançados em IoT e Machine Learning

Mestrado e Doutorado em Telecomunicações

Profª. Victoria Dala Pegorara Souto

Inatel

O Paradigma do Aprendizado de Máquina



Introdução

- Neste tópico vamos explorar o que o **aprendizado de máquina** realmente é em um nível mais fundamental.

- Basicamente, o que vamos discutir é um **novo paradigma**, onde **ao invés de programar uma solução** para um determinado problema, vamos **ensinar um computador a aprender a solução** através de experiências prévias.

- Esse novo paradigma tem o potencial para **resolver problemas** que
  - **não podem ou**
  - **são muito difíceis de serem resolvidos programaticamente**.

Machine Learning

Programação Tradicional:

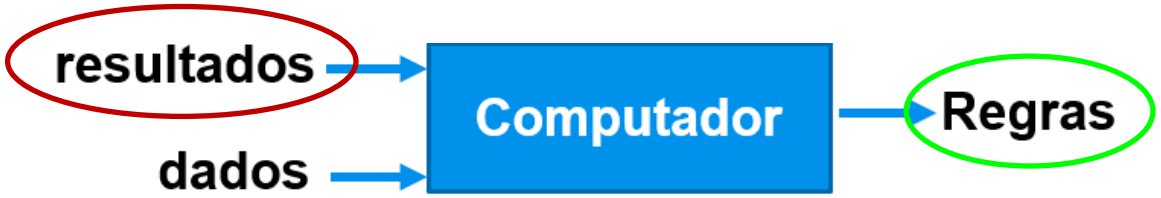
- Você dá as **regras** (i.e., o programa) e os **dados;**
- O computador gera o **resultado**.



→ **Exemplo:** Se o e-mail tiver a palavra 'PROMOÇÃO', mova para o lixo.

Machine Learning:

- Você dá os **dados** e os **resultados;**
- O computador descobre as **regras**.



→ **Exemplo:** Aqui estão 1.000 e-mails que são *spam* e 1.000 que não são. Descubra o padrão sozinho.

A máquina cria suas próprias regras baseadas nos dados.

Programação Tradicional



- Na programação tradicional, o **programador analisa um problema e cria um programa** para resolvê-lo.
  - **Programa:** sequência de regras que definem o comportamento do programa.
- Na sequência, o **computador recebe o código e os dados** (i.e., entradas), **o aplica aos dados e retorna os valores de saída**.
- Em outras palavras, na programação tradicional, o **programador cria as regras** (i.e., programa) que **mapeiam as entradas nas saídas**.

Programação Tradicional



- Porém, em alguns casos é muito difícil criar um código para solucionar um problema de forma geral (e.g., detecção de objetos, tradução, etc.).
- **Vamos ver uma situação onde esse paradigma não funciona bem.**

Programação Tradicional



```
if(speed<4){
    status=WALKING;
}
```



```
if(speed<4){
    status=WALKING;
} else {
    status=RUNNING;
}
```



```
if(speed<4){
    status=WALKING;
} else if(speed<12){
    status=RUNNING;
} else {
    status=BIKING;
}
```



```
// ???
```

- Vamos supor que queremos criar uma aplicação para celular/*smart watch* que **detecte atividades** como **andar**, **correr** e **pedalar**.
- Nesse caso, o programador analisaria o problema e diria que podemos usar a **velocidade** e criar algumas **regras** para **diferenciar as atividades**.

→ Mas e se quisermos estender a aplicação para identificar que o usuário está **jogando golfe**, como poderíamos identificar essa atividade?

Programação Tradicional



```
if(speed<4){
    status=WALKING;
}
```



```
if(speed<4){
    status=WALKING;
} else {
    status=RUNNING;
}
```



```
if(speed<4){
    status=WALKING;
} else if(speed<12){
    status=RUNNING;
} else {
    status=BIKING;
}
```

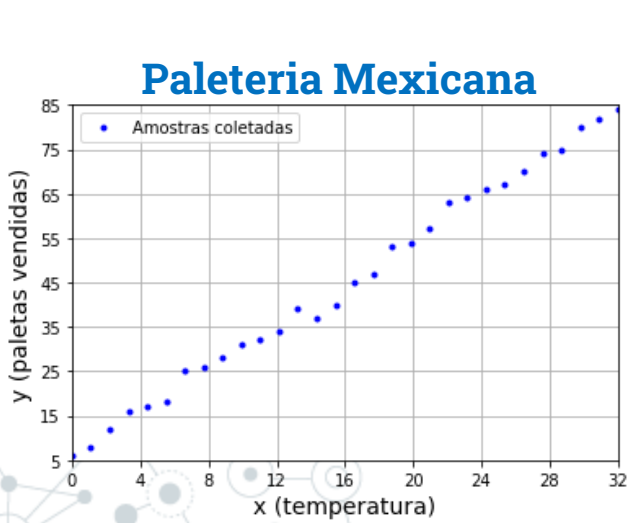


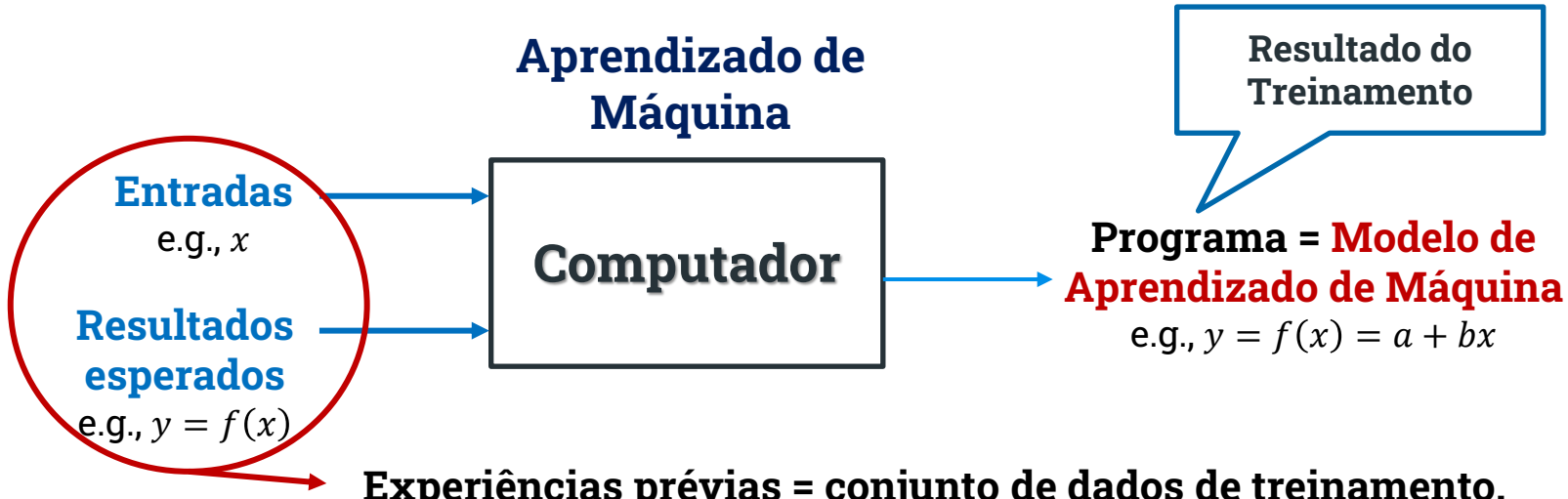
```
// ???
```

- Além disso, vejam que as **regras** que criamos são bem **simples e com certeza gerariam vários erros de identificação**.
  - Por exemplo, podemos correr ladeira abaixo mais rápido do que pedalamos ladeira acima.
- **Aprendizado de máquina** pode nos ajudar a resolver este problema.

Machine Learning

- **"... aprender sem serem explicitamente programados."**
- Esse trecho pode ser entendido se reorganizarmos a figura anterior.
  - E se ao invés de tentarmos **nós mesmos definir as regras** **que agem sobre os dados para gerar as respostas**, fizermos o **contrário**?

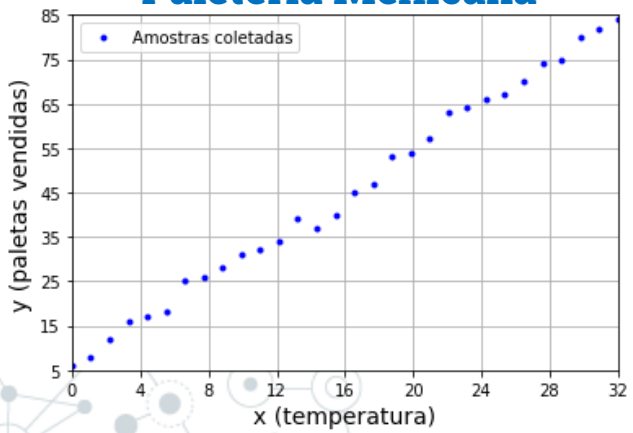


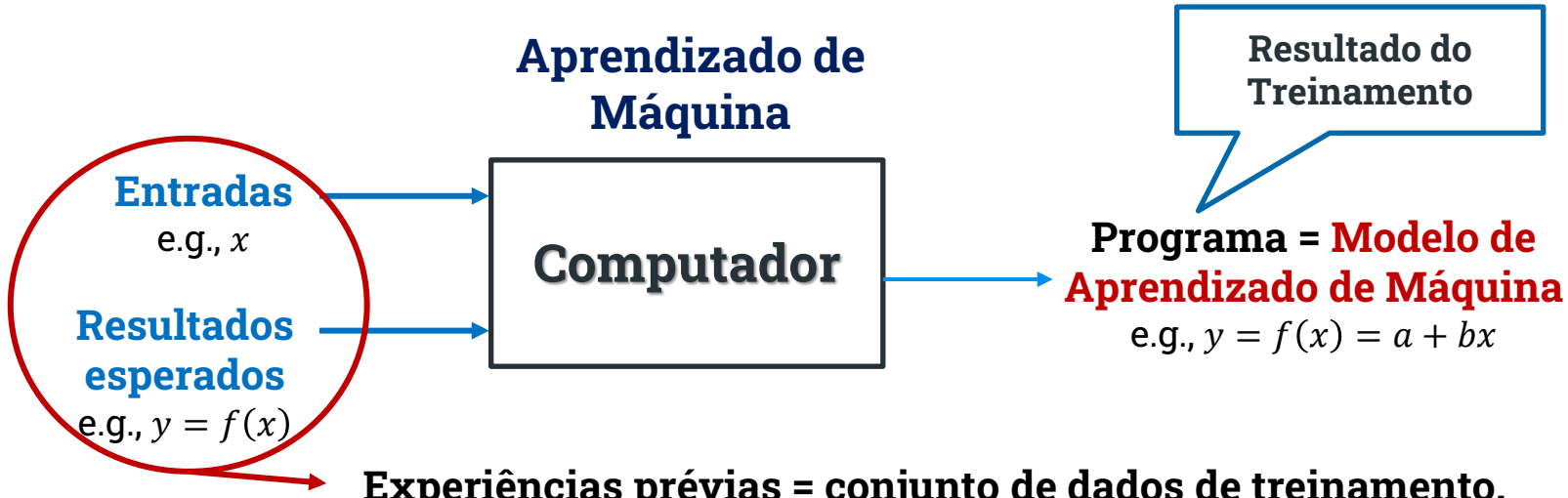


Machine Learning

- Ou seja, **fornecer as respostas e os dados (i.e., entradas) a um computador e deixar que ele descubra as regras** que geram as saídas.
- Queremos que o computador **aprenda** o mapeamento das entradas nas saídas.

Paleteria Mexicana

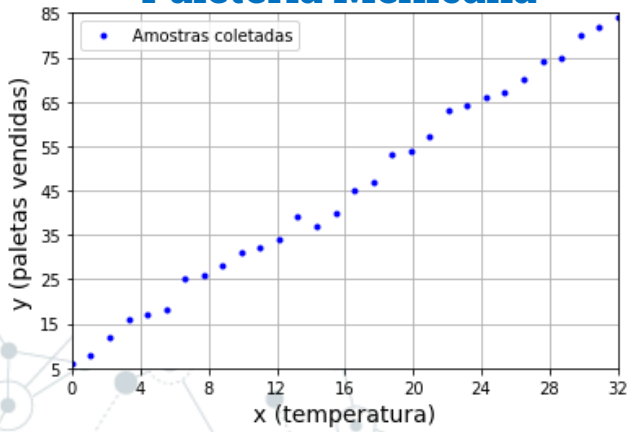


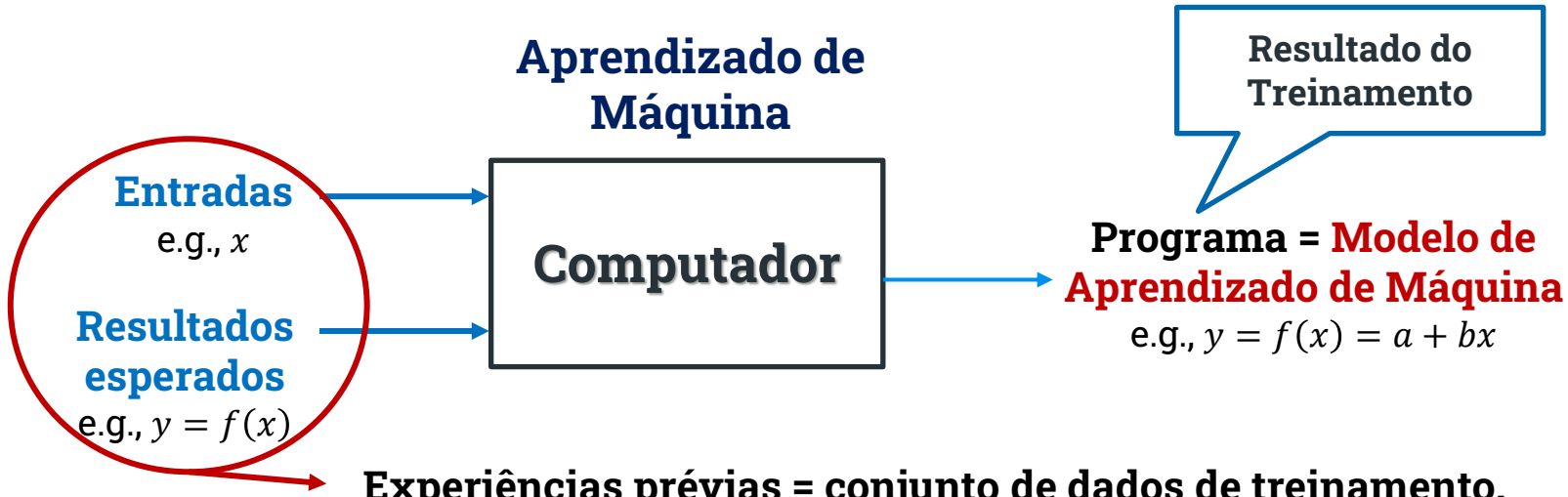


Machine Learning

- **Como a máquina apende?**
  - Através de treinamento com um **conjunto de experiências prévias** (chamado de **conjunto de treinamento**).
  - Durante o treinamento, o **conjunto de treinamento** é **apresentado ao modelo diversas vezes** (chamada de **iterações**).
  - Com isso, o modelo vai **iterativamente se atualizando** e **aprendendo** um **bom mapeamento** das entradas nas saídas esperadas.

Paleteria Mexicana





Machine Learning

- No caso da nossa aplicação, podemos **coletar informações de diferentes sensores** (i.e., entradas) e **rotulá-las** (i.e., saídas esperadas) com a atividade do usuário.

- Usando este **conjunto de dados** (i.e., dados dos sensores e rótulos), o computador pode **descobrir regras que identificam as atividades**:
  - caminhar,
  - correr,
  - pedalar e
  - até mesmo jogar golfe.

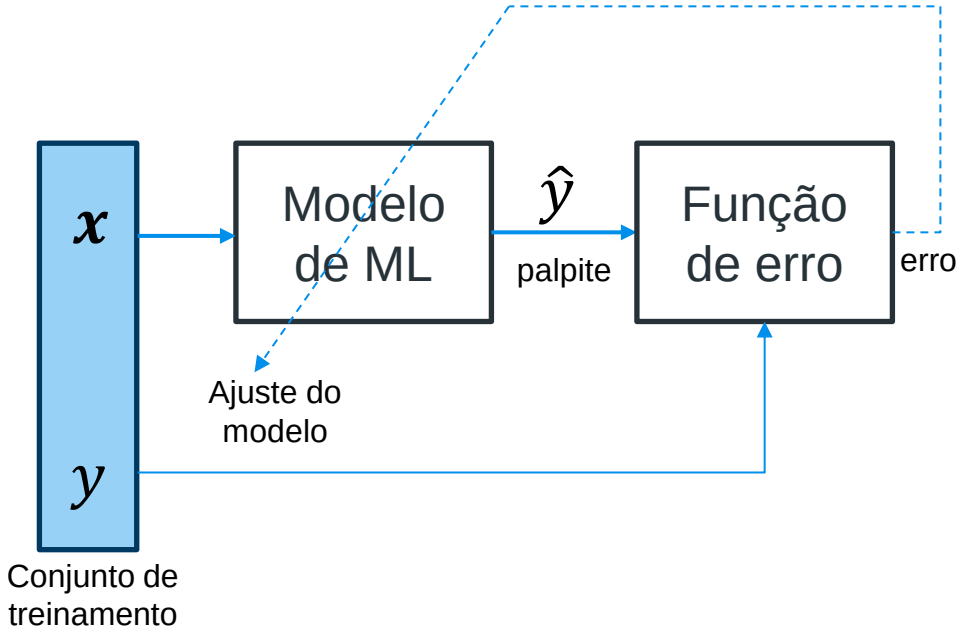





01010010101001010101 10010101010010111101 01001010100101001001 01010010101001010101	10101001010010101010 01010100100100100001 00100111110101001111 10101001001111010111	10010100111110101010 11010101110101011110 10101011110101001011 11111100011110101010	11111111110100111010 00111110101111101010 01011101010101001110 10101010101001111110
Label = WALKING	Label = RUNNING	Label = BIKING	Label = GOLFING

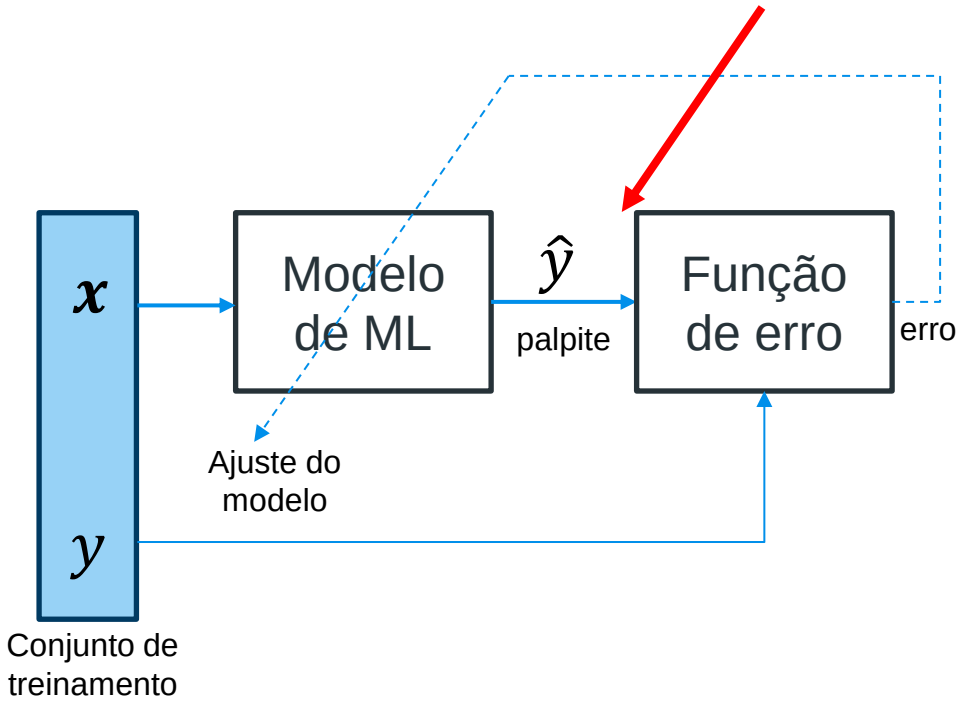
→ O computador aprende, através de **treinamento**, **padrões** nos dados de diferentes sensores que **podem ser mapeados nas atividades**.

Machine Learning



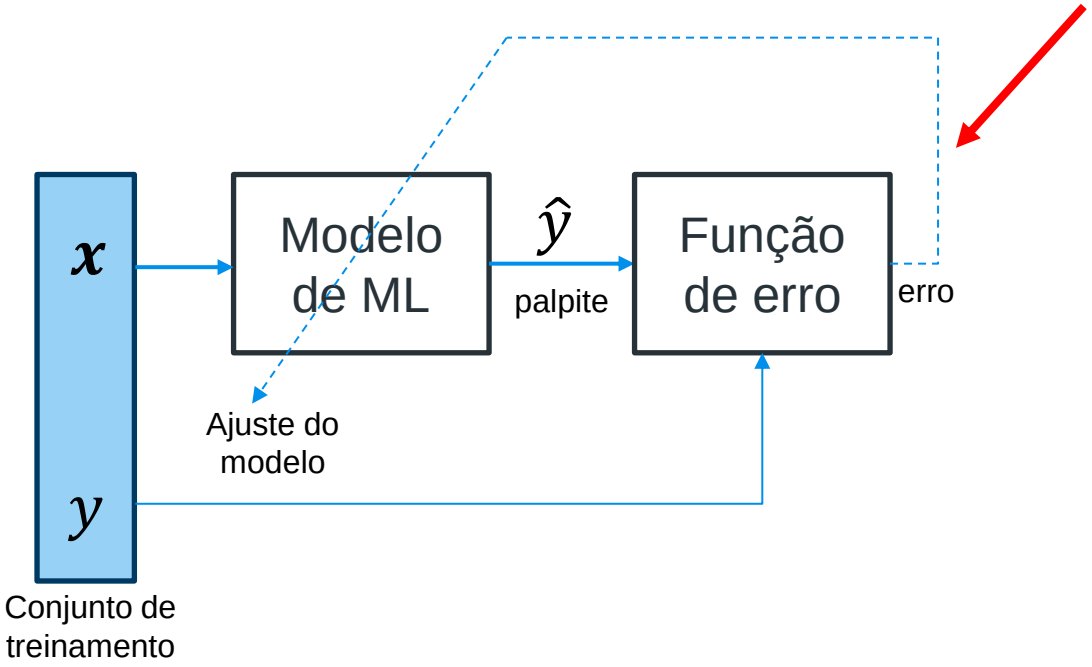
- Na verdade, nós não treinamos a máquina, mas sim, um **modelo**.
  - Por exemplo, um neurônio artificial.
- Vamos ver como o treinamento dos modelos funciona em alto nível.
- Essa explicação em alto nível nos **dará a base** para o entendimento do **algoritmo** por trás do **treinamento de muitos modelos de ML**.

Machine Learning



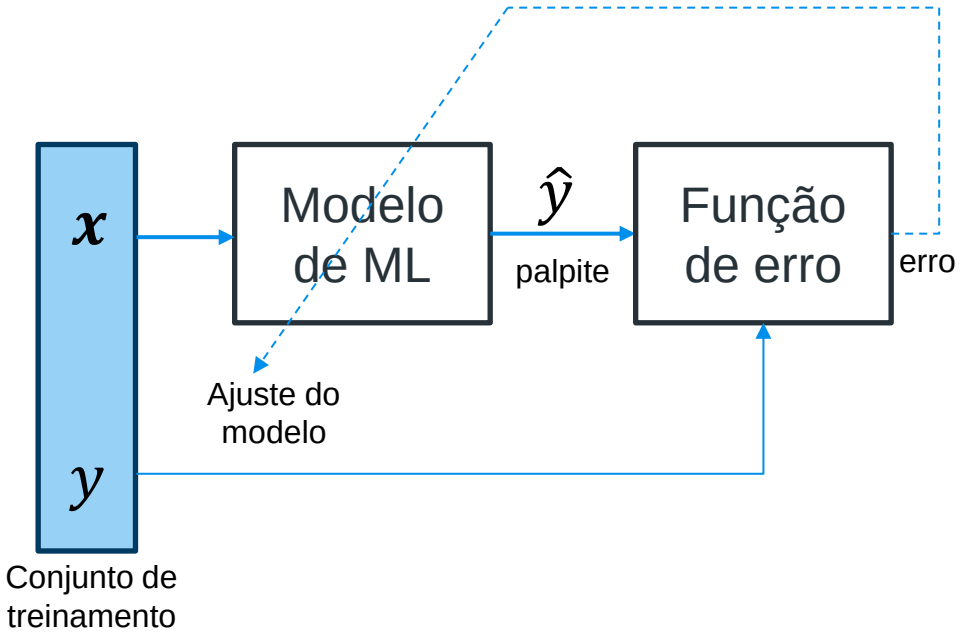
- Primeiro, o modelo de ML faz um **mapeamento aleatório** da entrada, **x**, em um valor de saída, $\hat{y}$.
  - Por exemplos, os pesos sinápticos e de bias de um neurônio são inicializados de forma aleatória.
- Ou seja, o modelo dá um **palpite** sobre qual deve ser a saída para aquela entrada.
- Em seguida, usando as saídas esperadas, y, (i.e., **rótulos**), **mede-se o quão bom ou ruim foi esse palpite**.
- Medimos a **qualidade** do palpite usando uma função chamada de **função de perda, erro ou custo**.

Machine Learning



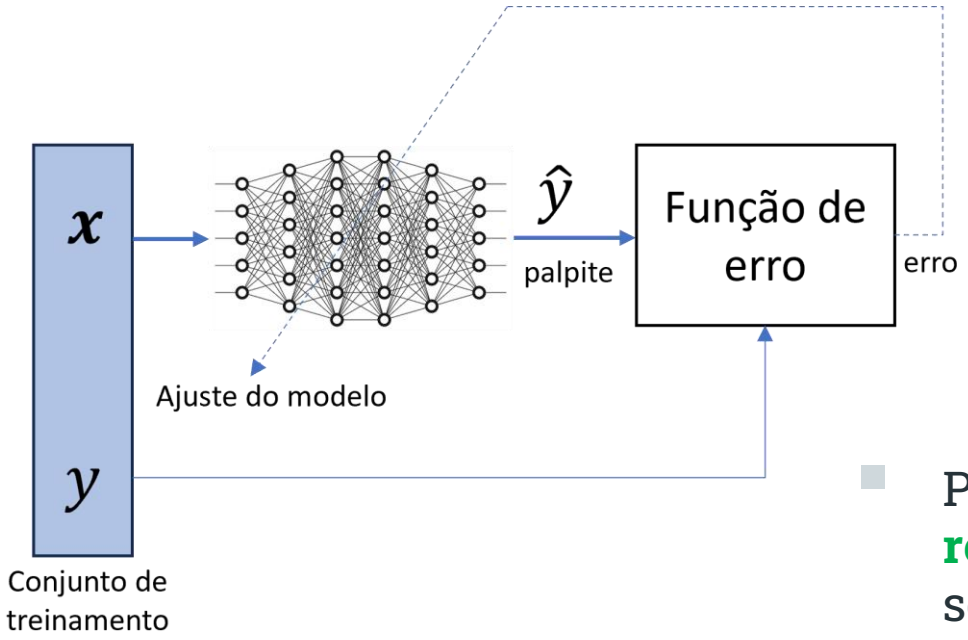
- Na sequência, o **erro** é usado para **otimizar o modelo** e, com isso, **melhorar o próximo palpite**.
  - **Por exemplo, atualizar os pesos do neurônio.**
- Como veremos, a informação fornecida pelo **erro** é de **suma importância para o treinamento de modelos de aprendizado supervisionados**.
- O **aprendizado supervisionado** é uma abordagem do aprendizado de máquina em que o **modelo** aprende a partir de **dados rotulados**.
- Ou seja, cada exemplo do conjunto de treinamento contém uma entrada e a saída esperada (ou rótulo).

MACHINE LEARNING



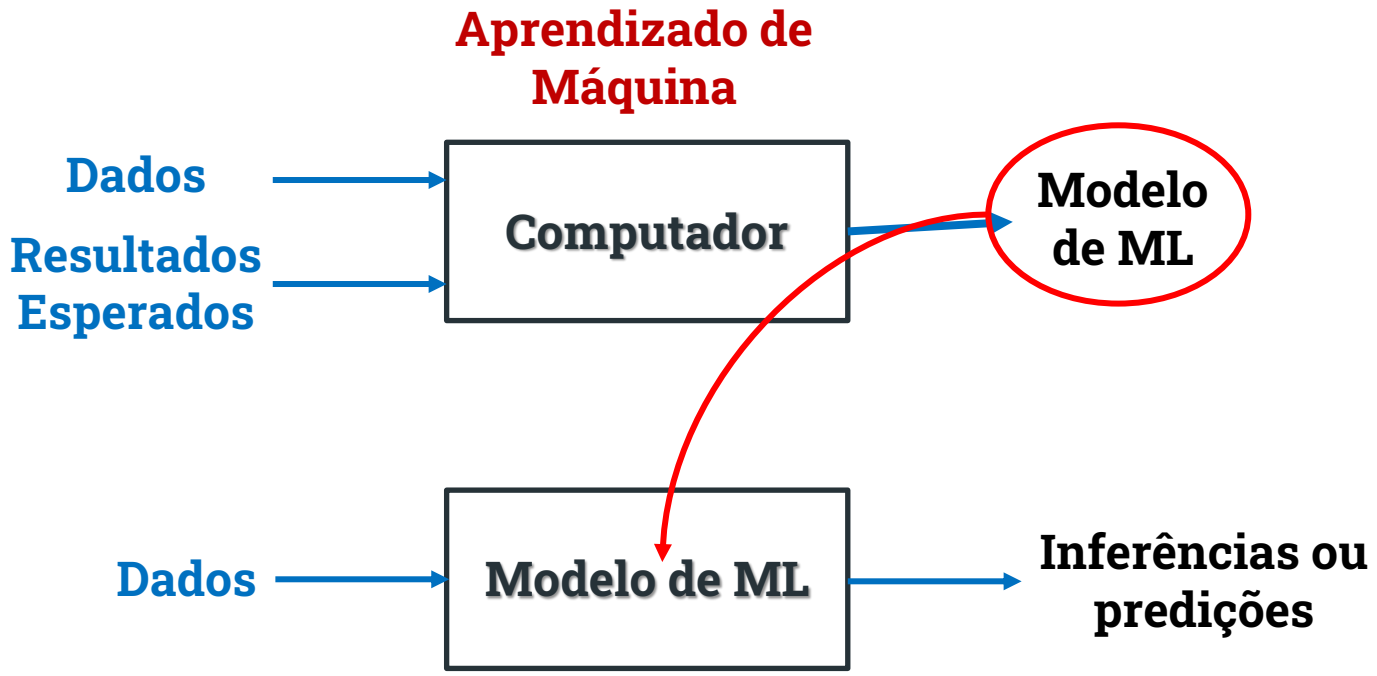
- Esse processo iterativo de aprendizado é repetido até que o erro seja **minimizado**.
- A **ideia** é que a cada **repetição** (ou **iteração**), o **novo palpite** se torne **melhor do que o anterior**, fazendo com que o **erro diminua** e o **modelo se torne mais preciso**.
- Percebam que a **solução** para o problema é **encontrada com base em experiências prévias**, ou seja, com o conjunto de treinamento (i.e., entradas, **x**, e saídas esperadas, y).
- Isso é chamado de **raciocínio indutivo**, que é um processo pelo qual chega-se a **conclusões gerais** a partir de **experiências passadas**.

MACHINE LEARNING



- Por exemplo, o modelo de ML pode ser um **neurônio** ou uma **rede neural** onde o **processo de treinamento** **atualiza** os seus **pesos** (sinápticos e de bias) a fim de **minimizar o erro** e, consequentemente, **resolver problemas classificação** (e.g., de imagens), **regressão** (i.e., aproximação de curvas), etc.

INFERÊNCIA



- Após o treinamento do modelo, o usamos para fazer **inferências**.

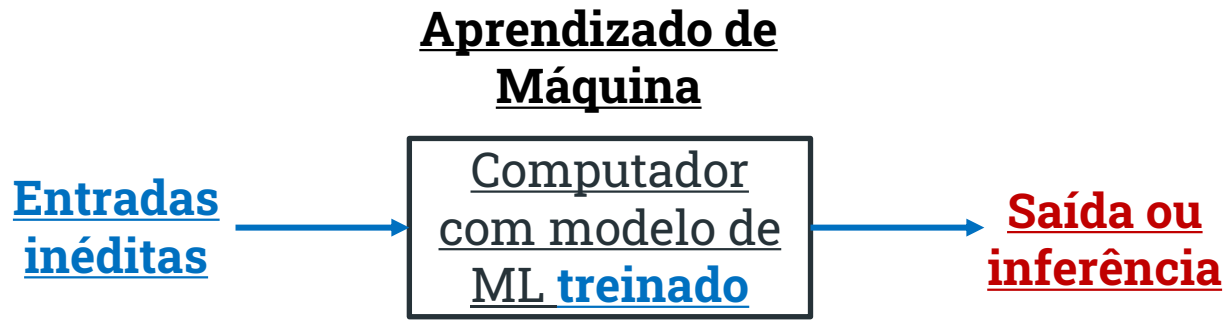
Ou seja, o usamos para fazer **predições** (i.e., valores de saídas) **para entradas inéditas**.

GENERALIZAÇÃO

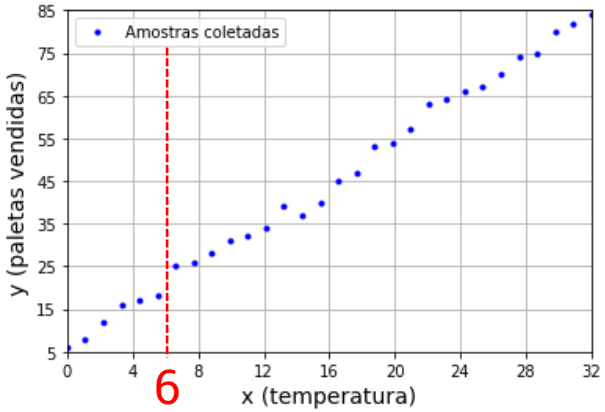
- Porém, não basta que o algoritmo de ML treine um modelo que faça um bom mapeamento **apenas para os dados do conjunto de treinamento**.

- O **algoritmo** de ML deve **treinar** um **modelo** que **aprenda** uma **solução geral**, ou seja, um modelo que **faça predições precisas para entradas não vistas durante o treinamento**.

- Essa extrapolação do conhecimento adquirido durante o treinamento é chamado de **generalização**.



GENERALIZAÇÃO

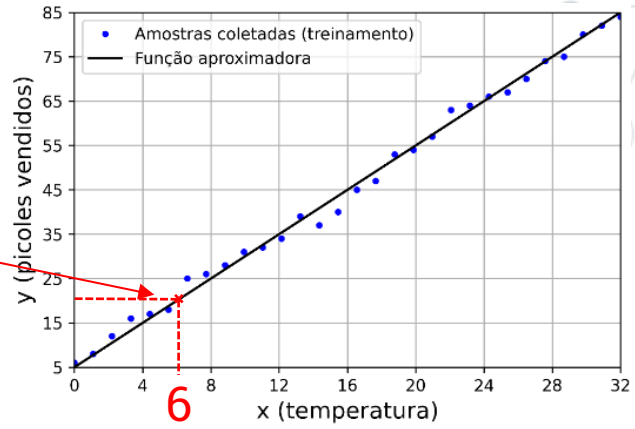


Qual é a estimativa de paletas vendidas quando a temperatura é de ≈ 6 graus (valor não visto durante o treinamento)?

Aprendizado de Máquina

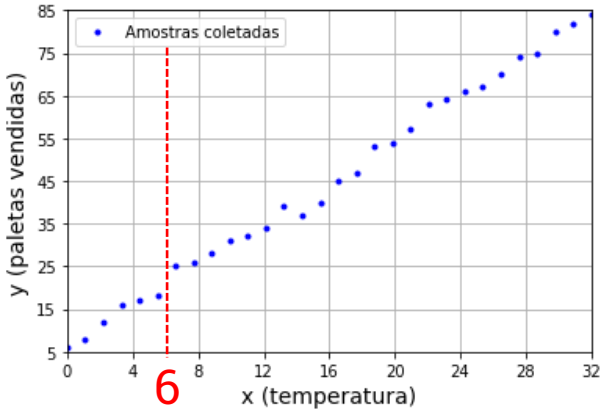
Entradas Inéditas → 6 → Computador com modelo de ML *treinado* → 21 → **Saída ou inferência**

O modelo aprendeu que uma **reta** é um **bom mapeamento.**



A partir do mapeamento aprendido (reta), o modelo gera como saída o valor 21, que é coerente com o restante dos dados.

GENERALIZAÇÃO



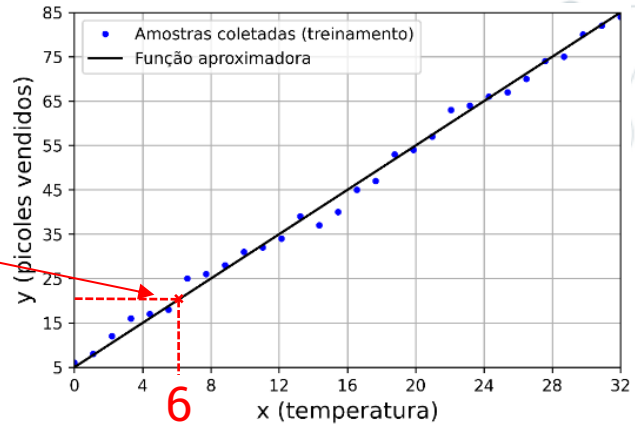
Qual é a estimativa de paletas vendidas quando a temperatura é de ≈ 6 graus (valor não visto durante o treinamento)?

Aprendizado de Máquina

Entradas Inéditas → 6 → Computador com modelo de ML *treinado* → 21 → **Saída ou inferência**

O modelo aprendeu que uma reta é um bom mapeamento.

No sentido da minimização do erro E de ter uma boa capacidade de generalização.



A partir do mapeamento aprendido (reta), o modelo gera como saída o valor 21, que é coerente com o restante dos dados.

PARADIGMAS DE APRENDIZADO



Inatel

Precisão de um Modelo de ML



Introdução

- Neste tópico vamos ver como **medir o desempenho** de um **modelo de aprendizado de máquina** ao longo do seu processo de aprendizagem.
- Para isso, como já discutido brevemente antes, usaremos uma função chamada de **função de erro** ou de **perda**.
- Idealmente, o **processo de treinamento** tem como **objetivo minimizar o erro** e, consequentemente, **melhorar o desempenho do modelo**.
- Além disso, veremos em breve **um algoritmo para minimizar o erro**.
- Porém, primeiro, vamos aprender como calcular o erro.

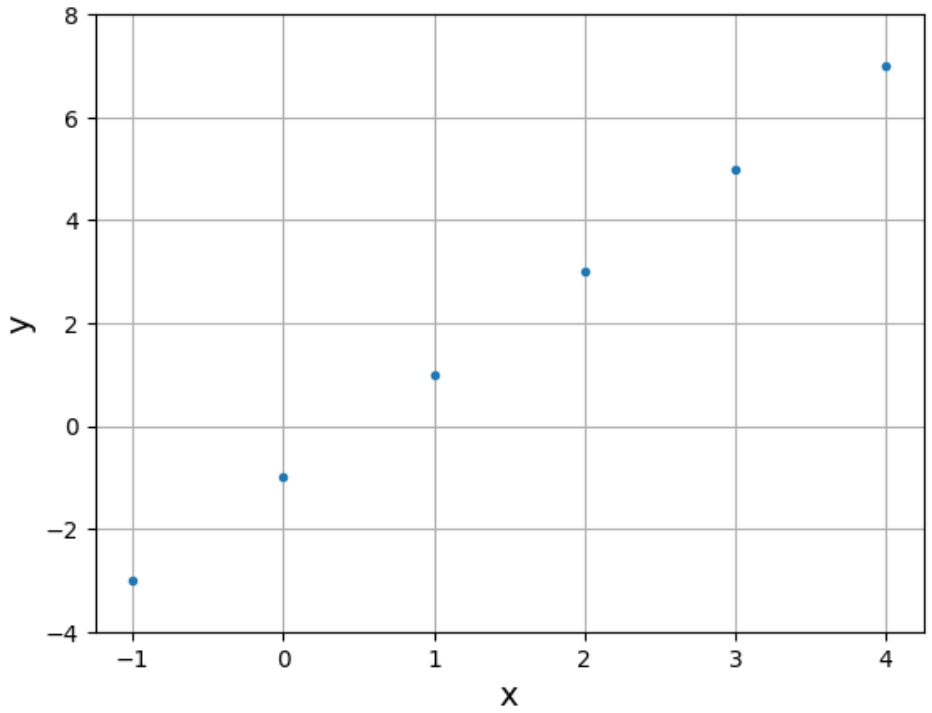
Mapeando x em y

$$x = \{-1, 0, 1, 2, 3, 4\}$$
$$y = \{-3, -1, 1, 3, 5, 7\}$$

- Considerem esses dois conjuntos de números.
  - Entradas e suas respectivas saídas esperadas.
- Qual a relação entre os dois conjuntos?
  - **Ou seja, qual é a função que mapeia os valores de x em y?**
- Nós sabemos que y é uma função de x, nós só não sabemos que função é essa.
- **Que tal plotarmos esses pontos?**

Mapeando x em y : Função Hipótese

$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$



- Ao plotarmos os pontos, percebemos que existe uma **relação linear** entre eles.
- Podemos **hipotetizar** que uma **reta** explica esse mapeamento.
- Portanto, usamos a função de uma reta para definir o mapeamento:

$$y = a_0 + a_1 x.$$

- Agora precisamos encontrar os valores dos parâmetros (ou pesos) a_0 e a_1.

Suposição e Predição

$$\hat{y}[n] = h(x[n]) = -1 + 3x[n]$$

→ $\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$

→ **Predições feitas pela função hipótese atual:**

$$\hat{\boldsymbol{y}} = \{-4, -1, 2, 5, 8, 11\}$$



- Vamos começar **atribuindo** alguns **valores aleatórios para os pesos** ($a_0 = -1$ e $a_1 = 3$), ou seja, vamos fazer uma **suposição** (dar um **palpite**) sobre os valores.

- **Como medimos se essa função hipótese é boa ou ruim?**
  - Usamos os valores de $x[n], \forall n$ para obter o mapeamento (i.e., predição) feito pela função e comparamos com os valores de saída esperados, $\mathbf{y}$.

Como Avaliar a Função Hipótese?

$$\hat{y}[n] = h(x[n]) = -1 + 3x[n]$$

→ $x = \{-1, 0, 1, 2, 3, 4\}$

→ **Predições feitas pela função hipótese atual:**

$$\hat{y} = \{-4, -1, 2, 5, 8, 11\}$$

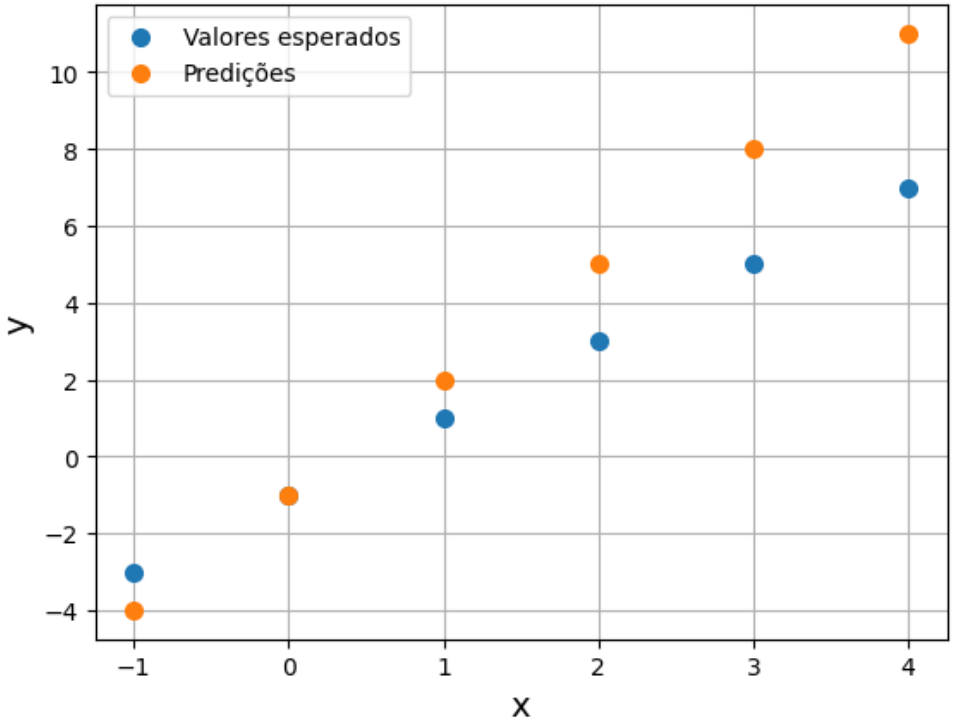
→ **Valores de saída esperados:**

$$y = \{-3, -1, 1, 3, 5, 7\}$$



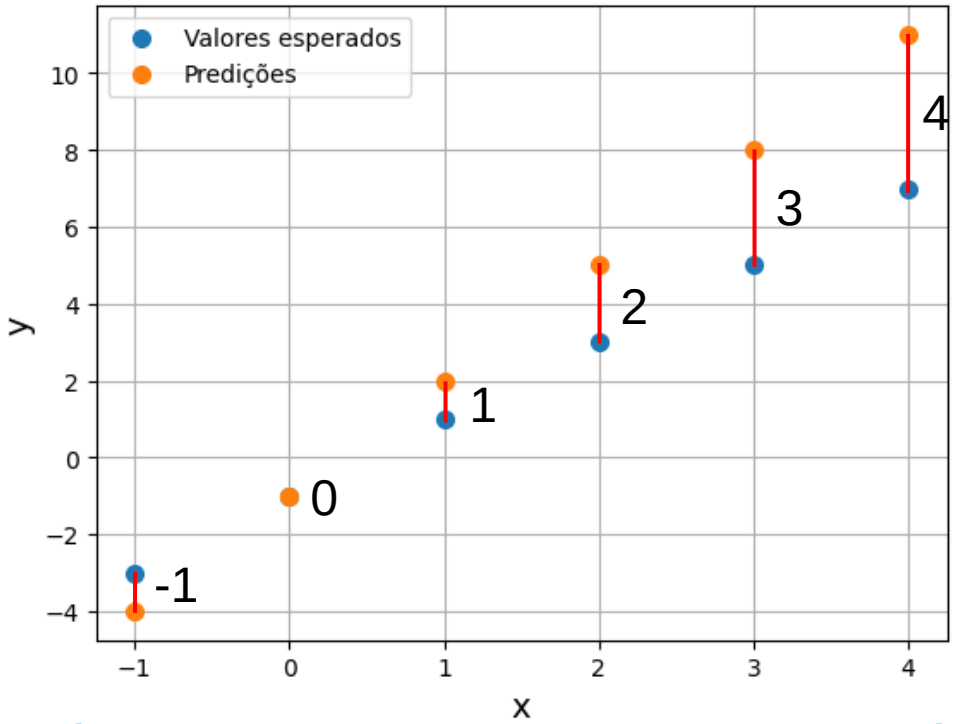
- Vemos que os **valores** preditos e esperados **não são os mesmos**.
- Os três primeiros valores até são próximos, mas os últimos três já estão mais distantes.
- Existe uma maneira de **formalizarmos** um **cálculo do quão bom ou ruim** essa função hipótese e suas predições são?

Como Avaliar a Função Hipótese?



- Talvez nós possamos definir uma métrica de desempenho analisando o gráfico com os valores esperados e preditos.
- Na figura ao lado, temos os pontos preditos e esperados para cada valor de x.

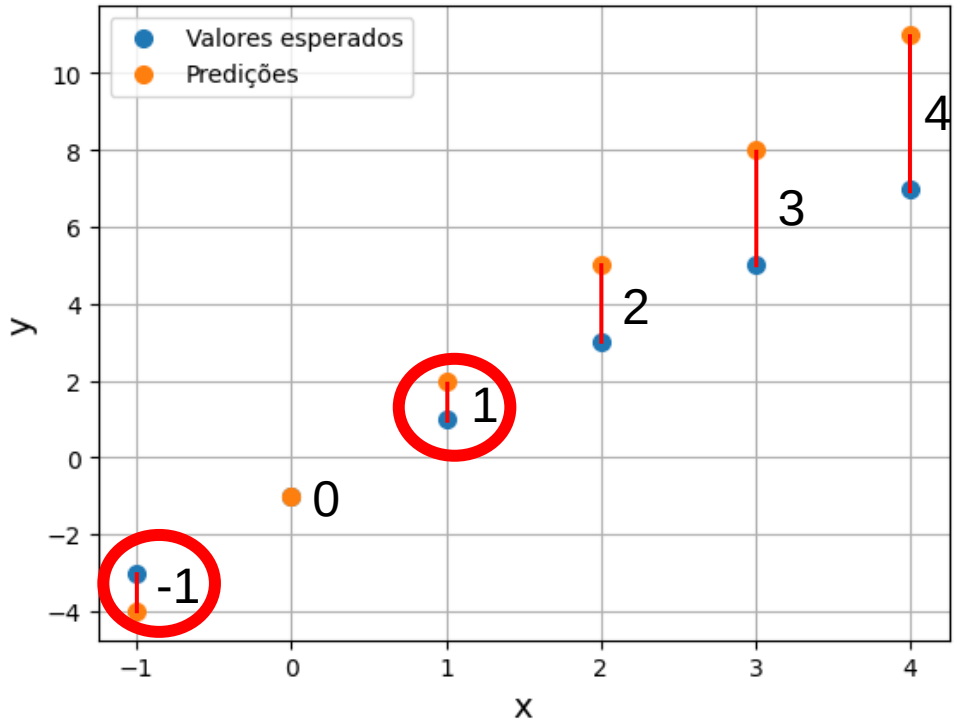
Como Avaliar a Função Hipótese?



$$\text{diff}[n] = \hat{y}[n] - y[n]$$

- Podemos traçar uma reta entre cada ponto e, pelo comprimento dessas retas (ou **diferença/distância entre os pontos**), **descobrir se a função hipótese é boa ou não**.
- O comprimento de cada reta é mostrado ao lado delas.
- Talvez nós possamos **calcular a média dos comprimentos** para obter o **erro médio** causado pela função hipótese atual.

Como Avaliar a Função Hipótese?



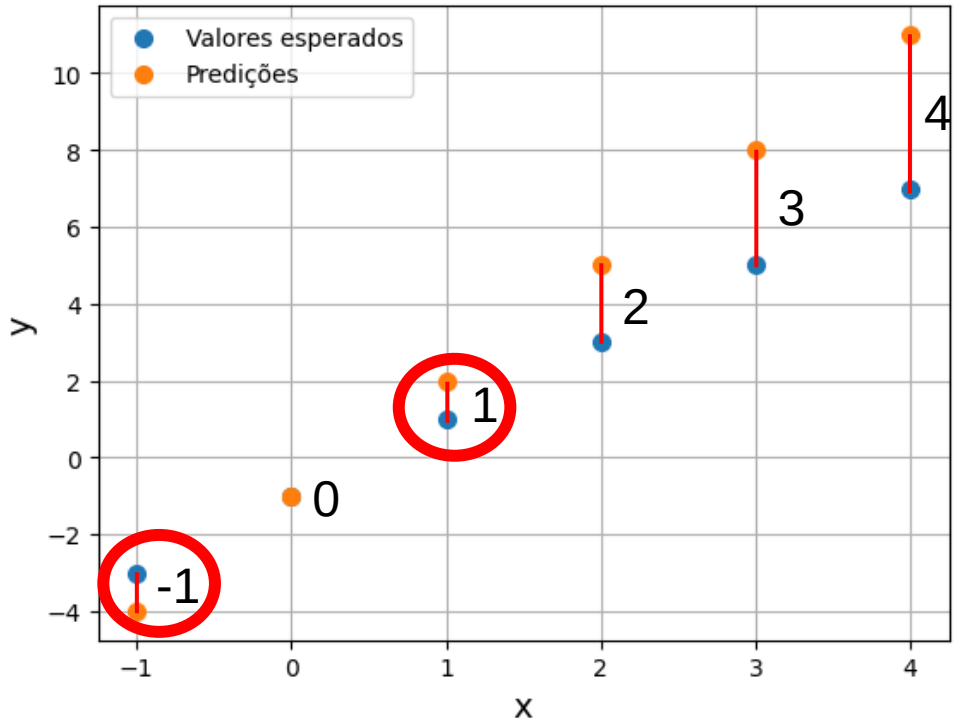
$$\text{erro médio} = \frac{1}{6}\sum_{0}^{5} \text{diff}\,[n]$$

$$\text{erro médio} = \frac{-1 + 0 + 1 + 2 + 3 + 4}{6} = 1.5$$

→ Temos um problema!

- Ao somarmos as diferenças, **valores negativos podem reduzir ou até mesmo anular valores positivos**.
- Mesmo as predições para os dois pontos, $x = -1$ e 1, estando erradas, seus **erros se cancelariam**, afetando a medida de desempenho.
- Isso poderia sugerir que 3 de 6 predições estão corretas, o que sabemos não ser verdade.
- O que podemos fazer para resolver esse problema?

Como Avaliar a Função Hipótese?



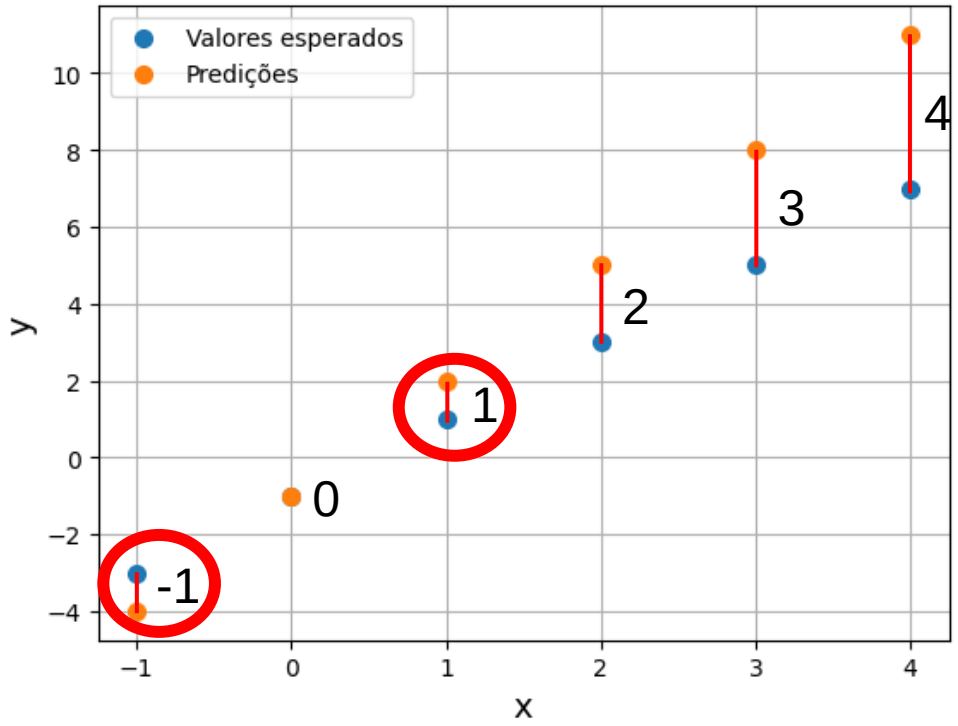
→ **Temos um problema!**

- Podemos **elevar ao quadrado todos os comprimentos**, fazendo com que todos eles sejam positivos e não se cancelem mais.
- Isso não afeta nossa métrica, pois aplicamos a todos os comprimentos.
- Essa métrica é chamada de **erro quadrático médio (EQM)**.

$$\text{erro médio } = \frac{1}{6}\sum_{0}^{5} \text{diff}\,[n]$$

$$\text{erro médio } = \frac{-1 + 0 + 1 + 2 + 3 + 4}{6} = 1.5$$

Como Avaliar a Função Hipótese?



→ **Temos um problema!**

- Usando o EQM, vemos que **nossa função hipótese não é boa**, pois o **valor ainda está longe de zero**, que é o menor valor possível e nosso objetivo final.

O que devemos fazer?

$$\text{erro médio} = \frac{1}{6}\sum_{0}^{5} \text{diff}\,[n]$$

$$\text{erro médio} = \frac{-1 + 0 + 1 + 2 + 3 + 4}{6} = 1.5$$

Otimizando a Suposição

$$\hat{y}[n] = h(x[n]) = -2 + 2x[n]$$

→ $\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$

→ **Predições feitas pela função hipótese atual:**

$$\hat{\boldsymbol{y}} = \{-4, -2, 0, 2, 4, 6\}$$

→ **Valores de saída esperados:**

$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$

→ **Diferença entre predições e valores esperados:**

$$\mathbf{diff}^2 = \{1, 1, 1, 1, 1, 1\}$$



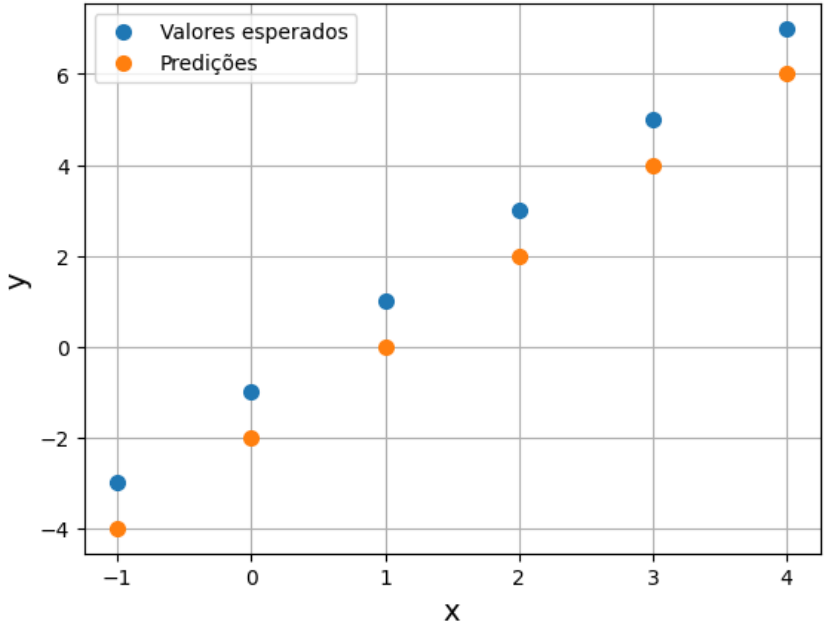
- Vamos supor outros valores para a_0 e a_1 e fazer predições com esta nova função hipótese.
- O novo palpite para os valores dos pesos será $a_0 = -2$ e $a_1 = 2$.

Como Avaliar a Função Hipótese?

$$\hat{y}[n] = h(x[n]) = -2 + 2x[n]$$

$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$

$$\text{EQM} = \frac{1 + 1 + 1 + 1 + 1 + 1}{6} = 1.0$$





- Já temos uma **função melhor**, pois o erro caiu de 5.17 para 1.0.
- Estamos indo na direção correta!
- Comparando as predições com os valores esperados, notamos que as **retas são paralelas**, sendo a única diferença um **deslocamento**.

Como Avaliar a Função Hipótese?

→ Nova Suposição

$$\hat{y}[n] = h(x[n]) = -1 + 2x[n]$$

$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$

→ **Predições feitas pela função hipótese atual:**

$$\hat{\boldsymbol{y}} = \{-3, -1, 1, 3, 5, 7\}$$

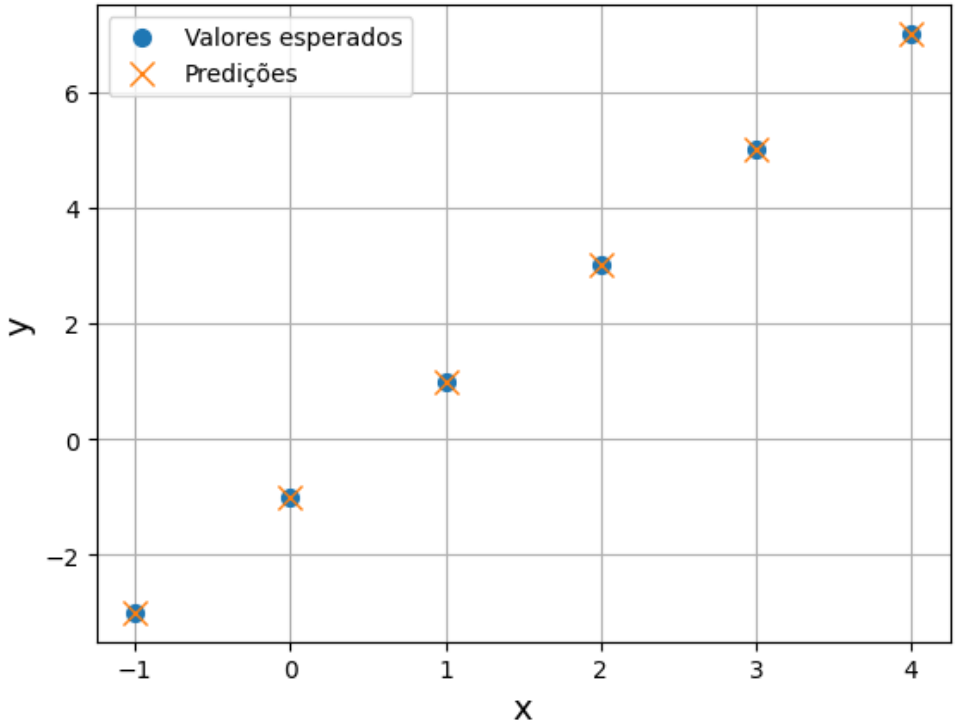
→ **Valores de saída esperados:**

$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$

→ **Diferença entre predições e valores esperados:**

$$\mathbf{diff}^2 = \{0, 0, 0, 0, 0, 0\}$$

$$\text{EQM} = \frac{0 + 0 + 0 + 0 + 0 + 0}{6} = 0.0$$



- Vamos **aumentar o valor de** a_0 para -1, fazer novas predições e calcular o erro.
- **Bingo!** Nossa nova suposição mapeia perfeitamente x em y, resultando em um EQM igual a 0.

Processo de Otimização

- O **processo iterativo de otimização** (ou **atualização**) dos pesos será sempre esse:
  - **Palpite** (i.e., pesos),
  - **Medida da qualidade** do palpite,
  - **Otimização do palpite** (i.e., atualização dos peso) **baseada na informação do erro**.
- As **iterações se repetem** até que algum **critério de parada** seja atingido: **número máximo de iterações**, **erro cair abaixo de um limiar**, etc.
- Esse é o processo por trás do algoritmo de otimização que veremos adiante.



Medidas de Erro

$$\text{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2}$$

$$\text{MAE} = \frac{1}{N}\sum_{n=0}^{N-1}|\hat{y}[n] - y[n]|$$

Usadas em problemas de aproximação de curvas

$$\text{CE} = -\frac{1}{N}\sum_{n=0}^{N-1}\sum_{i=0}^{Q-1} y_i[n]\log_e(\hat{y}_i[n])$$

Usada em problemas de classificação

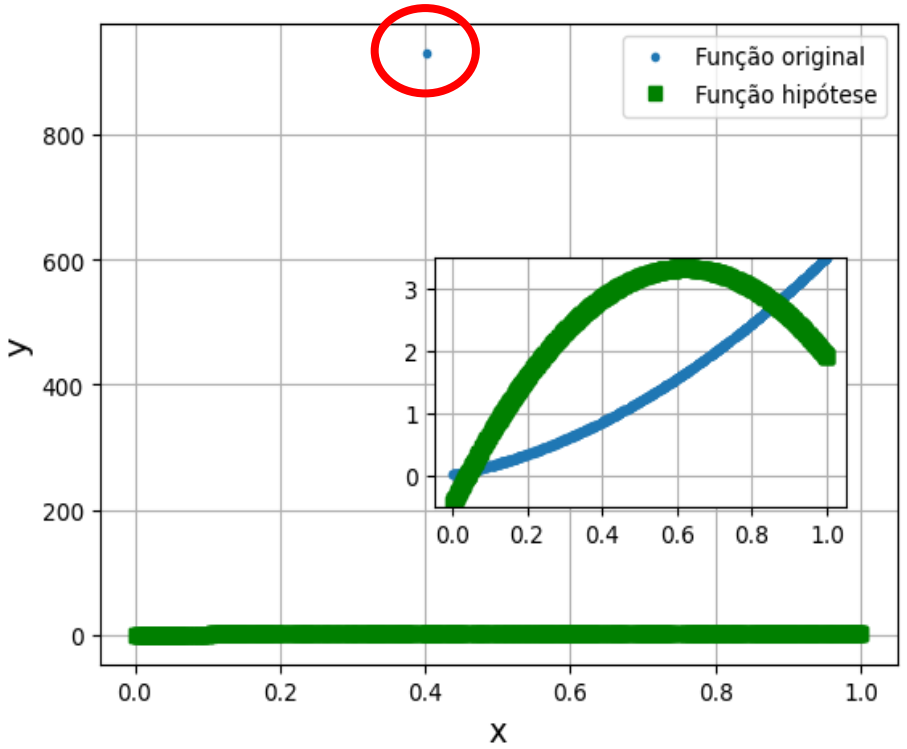
- Além do EQM (*Mean Squared Error* – MSE), existem diversas outras métricas de erro que podemos usar, como por exemplo:
  - **Raíz do Erro Quadrático Médio (*Root Mean Squared Error* – RMSE)**
  - **Erro Absoluto Médio (*Mean Absolute Error* – MAE);**
  - **Entropia Cruzada (*Cross-Entropy*);**
  - **Etc.**

Mean Squared Error – MSE

- É a função de erro **mais usada em problemas de regressão**.
- Calcula a média do **quadrado da diferença** entre os valores preditos e esperados.
- Assim, ele **penaliza mais (i.e., amplifica) erros maiores** (devido ao quadrado), o que pode ser útil se desejarmos que o modelo seja **mais sensível a esses erros**.

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

Mean Squared Error – MSE

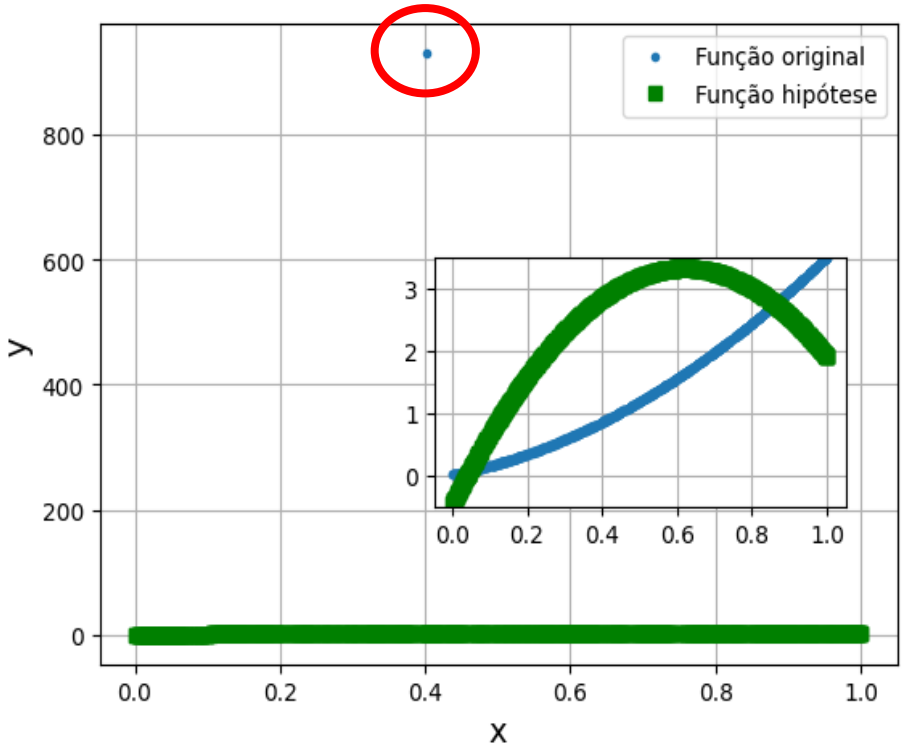


Função original: $y = 1 + x + 2x^2$

- Porém, como a diferença é elevada ao quadrado, ele **pode amplificar a influência de outliers**.
- **Outliers** podem levar o modelo a se concentrar demais em reduzir os erros grandes, em detrimento de um bom ajuste geral.
- A consequência disso é a **distorção da função hipótese**, fazendo com que ela se distancie de um mapeamento que represente o comportamento dos dados "normais".

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

Mean Squared Error – MSE



Função original: $y = 1 + x + 2x^2$

- Por usar o quadrado da diferença, **a métrica não fica na mesma escala dos dados originais**.
  - Por exemplo, se estivermos predizendo o preço de casas em dólares, os erros estarão em dólares^2.

$$\text{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

Root Mean Squared Error – RMSE

- Calcula a raiz quadrada do MSE, **retornando a métrica à mesma escala dos dados originais**.
- Isso é útil para interpretar o erro na mesma unidade do rótulo, o que pode facilitar a compreensão do impacto do erro.
- Assim como o MSE, **também amplifica erros maiores**.

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2}$$

Mean Absolute Error – MAE

- Calcula a **média das diferenças absolutas** entre as predições do modelo e os valores esperados.
  - Ou seja, **penaliza os erros de maneira uniforme**.
- Ao contrário do MSE e RMSE, não eleva os erros ao quadrado, o que o torna **menos sensível a outliers**.

$$\text{MAE} = \frac{1}{N}\sum_{n=0}^{N-1}|\hat{y}[n] - y[n]|$$

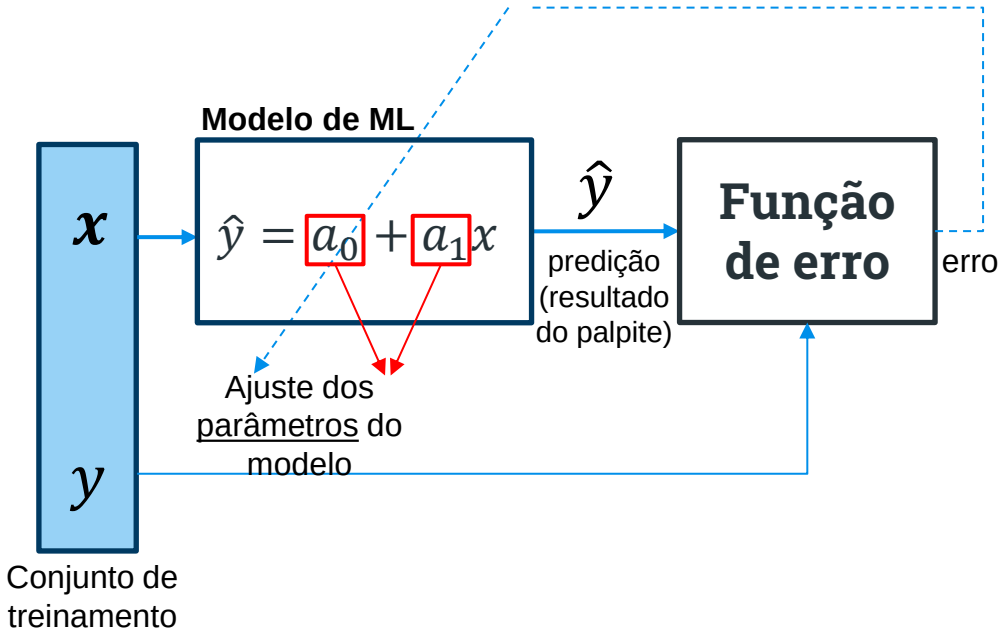
- **Representa os erros na mesma escala dos dados originais**, facilitando a interpretação.
- Boa opção quando desejamos um **erro mais uniforme em todas as predições** e desejamos **minimizar a influência de outliers**.

Mean Absolute Error – MAE

Qual devo usar?

→ O recomendável é **experimentar diferentes funções de erro** durante a **fase de desenvolvimento** para determinar qual delas se **alinha** melhor com os **objetivos e as características** específicas do seu **problema de regressão**.

Regressão ou Aproximação de Funções



- O problema que estamos vendo neste tópico é conhecido como **regressão** ou **aproximação de funções**.
- Nele, a função com formato de reta, que definimos como **função hipótese**, é o **modelo de ML** e o **objetivo da regressão é encontrar os parâmetros (ou pesos) que minimizam a função de erro**.
- **OBS**.: No exemplo que vimos nesse tópico, conseguimos **visualizar os dados** e **identificar a equação** (ou formato) da função hipótese facilmente.
- Porém, em outros casos, **além dos pesos**, temos que **descobrir o formato da função**.
- Veremos adiante formas de definir o melhor modelo para uma aproximação.

Regressão ou Aproximação de Funções

Ponto de Mínimo ou Solução Ótima

$$\text{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

$$= \frac{1}{N}\sum_{n=0}^{N-1}(a_0 + a_1 x[n] - y[n])^2$$

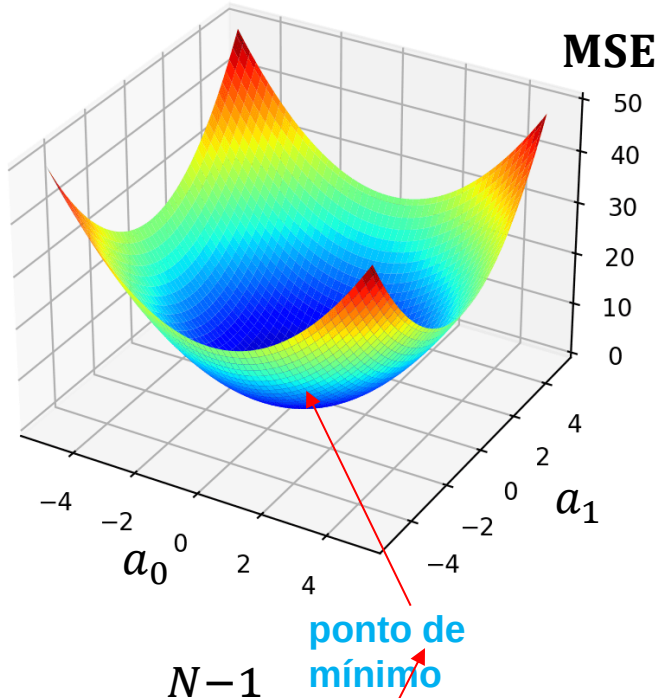
?

- Dado que o conjunto de treinamento é **conhecido e fixo**, notamos que a **função de erro** é **função dos pesos** do modelo.
- Assim, o **objetivo da regressão** é **encontrar o conjunto de pesos que minimiza a função de erro**.
- Esse é um **problema de minimização**.

→ Qual é o conjunto de pesos que resulta no menor erro possível?

Regressão ou Aproximação de Funções

Ponto de Mínimo ou Solução Ótima



$$\text{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(\hat{y}[n] - y[n])^2$$

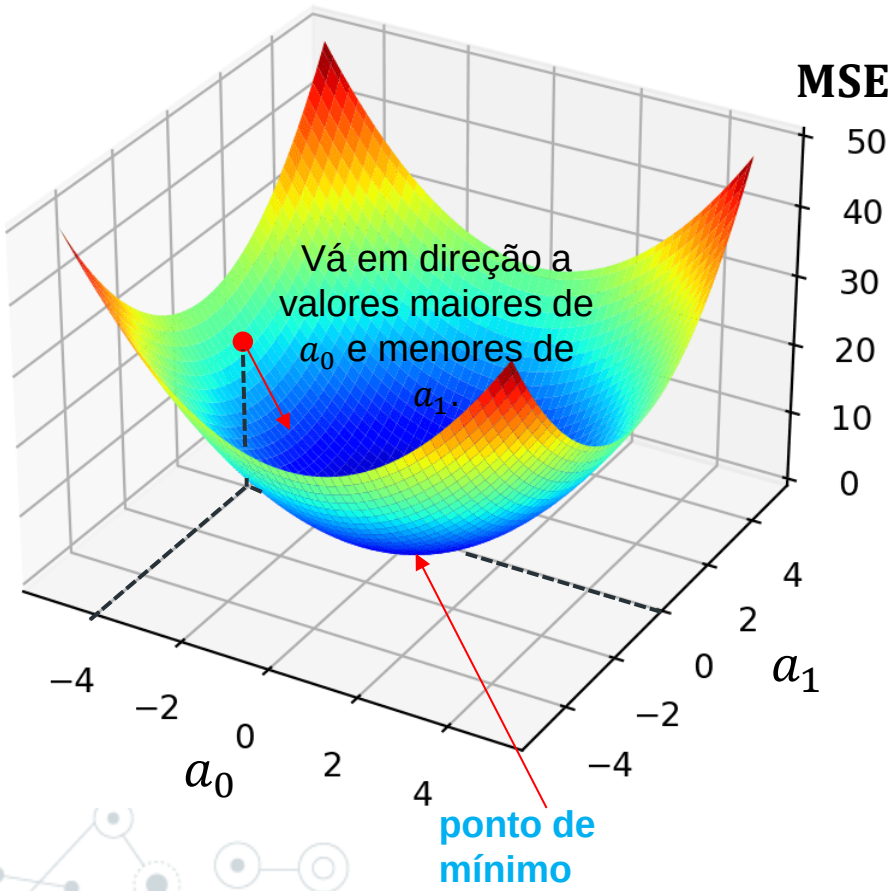
$$= \frac{1}{N}\sum_{n=0}^{N-1}(a_0 + a_1 x[n] - y[n])^2$$

?

- Se **plotarmos o erro em função dos pesos**, nós plotamos o que é conhecido como **superfície de erro**.
- Seu **ponto mais baixo** é chamado de **ponto de mínimo** e nos dá o **conjunto de pesos ótimo**, ou seja, os **pesos que minimizam a função de erro**.
- O algoritmo de otimização, que veremos em breve, **caminha** através da superfície sempre **indo em direção ao seu ponto mais baixo**.

Regressão ou Aproximação de Funções

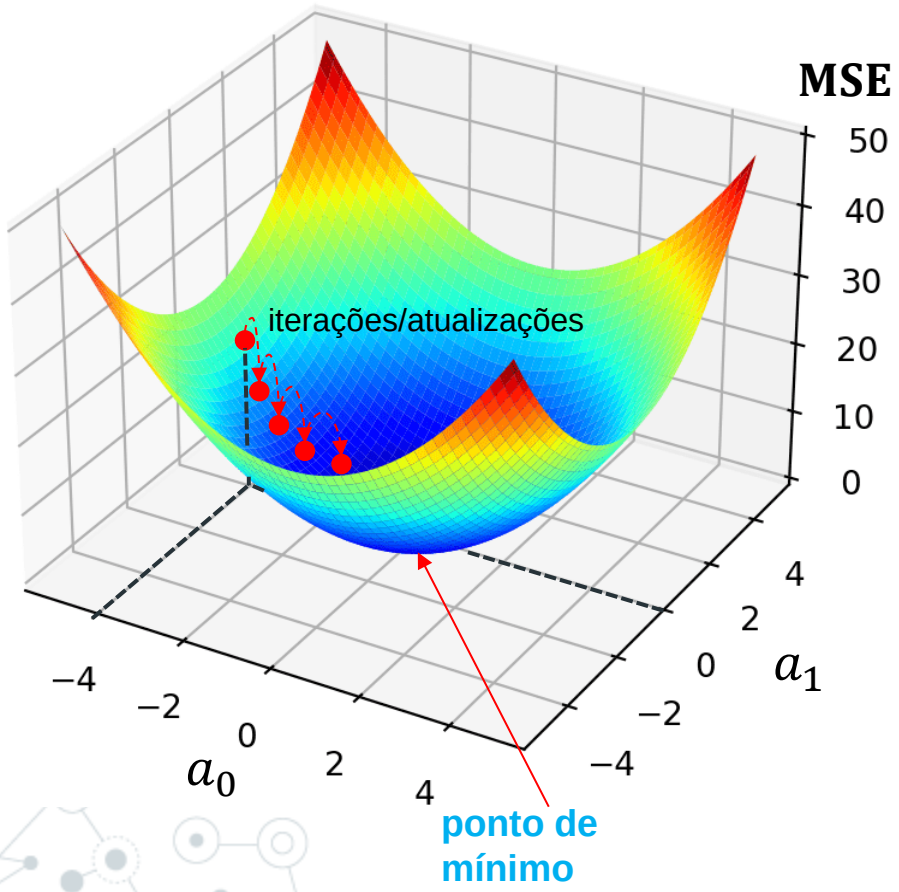
Otimização (Treinamento) do Modelo



- Esse **processo de busca** que tem como **objetivo** encontrar o **ponto** que **minimiza o erro** é chamado de **otimização** e, no contexto de ML, é conhecido como **treinamento do modelo**.
  - Treinamento é o processo de **atualizar o modelo** para obter predições melhores a cada **iteração**.
- O **treinamento** (ou atualização) do modelo se **baseia** na **informação obtida a partir da função de erro**.
- Como veremos, essa informação **aponta para a direção do ponto de mínimo** **(i.e., conjunto de pesos ótimo)**.

Regressão ou Aproximação de Funções

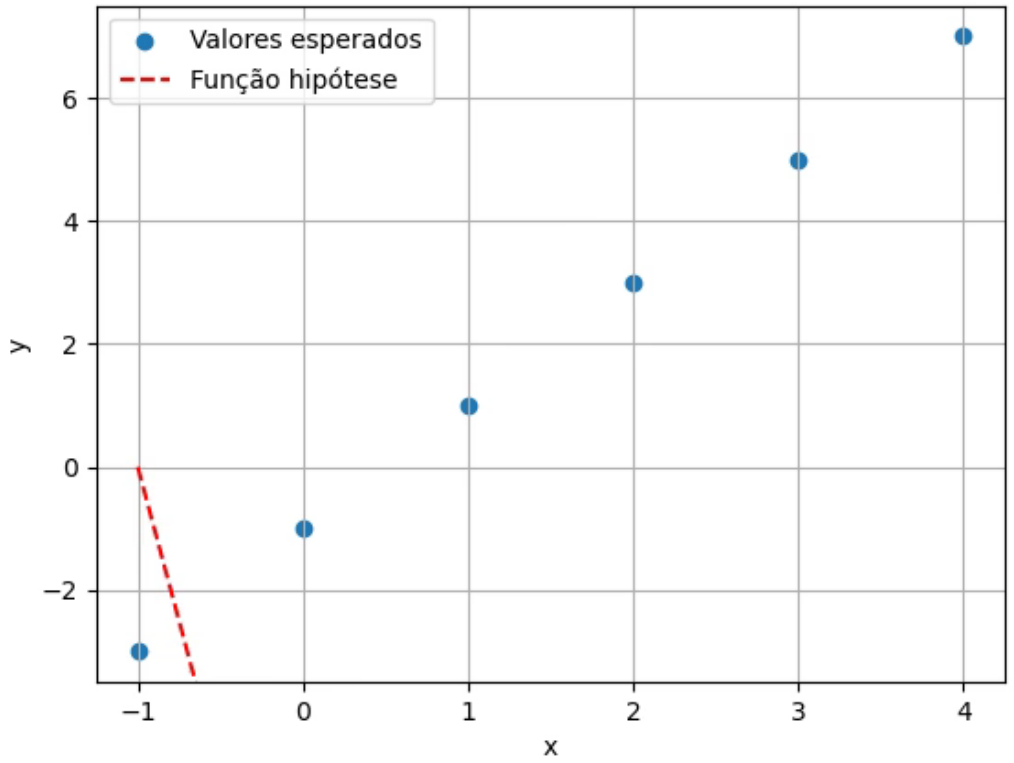
Gradiente Descendente



- Como veremos em breve, de posse dessa informação trazida pelo erro, o algoritmo de otimização, conhecido como **gradiente descendente (GD)**, consegue **caminhar iterativamente** através da **superfície de erro** até o **ponto de mínimo** (i.e., conjunto de pesos ótimo).

Regressão ou Aproximação de Funções

Gradiente Descendente



- A figura ao lado ilustra o processo iterativo de otimização realizado pelo GD.
  - A partir de um **ponto inicial** (i.e., uma **suposição aleatória** de pesos), o algoritmo **extrai a informação de direção do ponto de mínimo a partir da função de erro** e a usa para **atualizar o ponto atual** (i.e., otimizar a suposição atual).
  - Em geral, o **novo ponto** (i.e., conjunto de pesos atualizado) resulta em um **erro menor do que o anterior**.
  - Esse processo é repetido a partir do novo ponto até que algum critério de parada seja atingido.

Próximos passos

- Nesse tópico nós vimos o que é o **erro** (ou perda) e definimos uma forma de medi-lo.
- Agora, o próximo passo envolverá a **definição de um processo que utilize a informação do erro para gerar o próximo palpite** (ou suposição) de forma que ele **seja melhor do que o atual**.
- Esse processo é chamado de **otimização** e será abordado em breve.
- Mas primeiro, vamos dar uma olhada em um **exemplo** que nos dará uma ideia **sobre como funciona esse processo de otimização**.



Explorando a função de erro

Atividades

- Exercício: Encontre os pesos da função hipótese.

Inatel

Minimizando o Erro

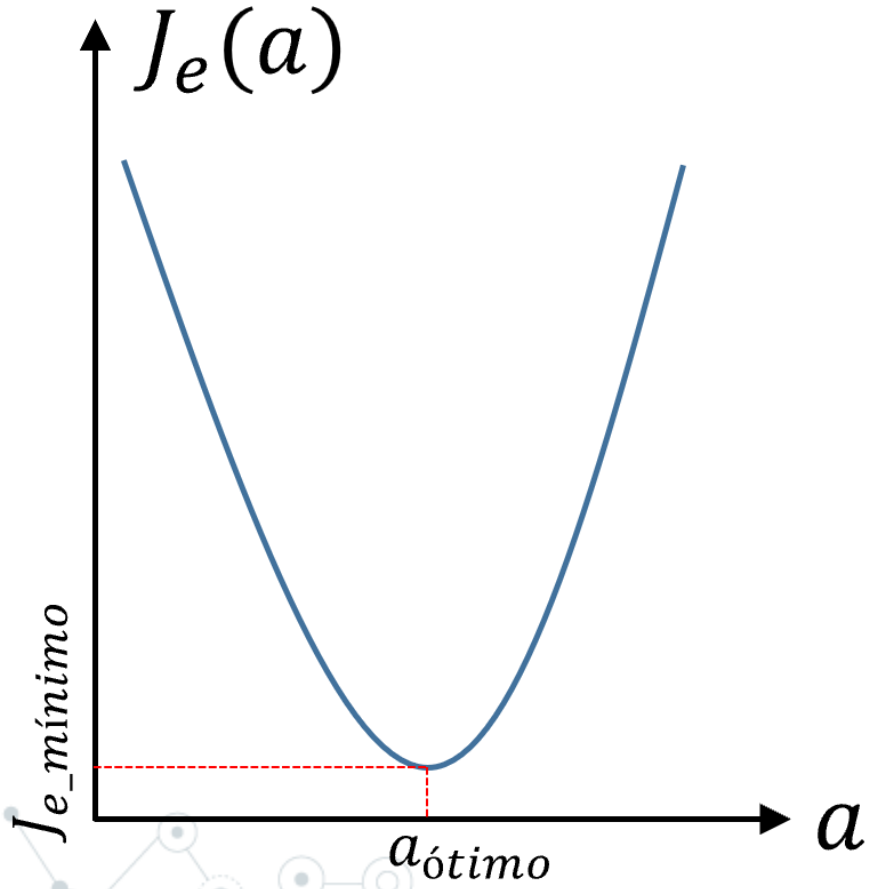


Introdução

- **Anteriormente, vimos como funciona o processo (*loop*) de treinamento.**
  - Damos um palpite.
  - Medimos a precisão desse palpite com a função de erro.
  - Então usamos a informação do erro para dar outro palpite, esperando que ele seja um pouco melhor do que o anterior.

- Em geral, **esse processo se repete até que o erro seja minimizado.**
- A ideia é que quanto menor o erro, mais preciso é o seu palpite.
- Portanto, neste tópico, exploraremos como minimizar o erro.

Superfície de Erro



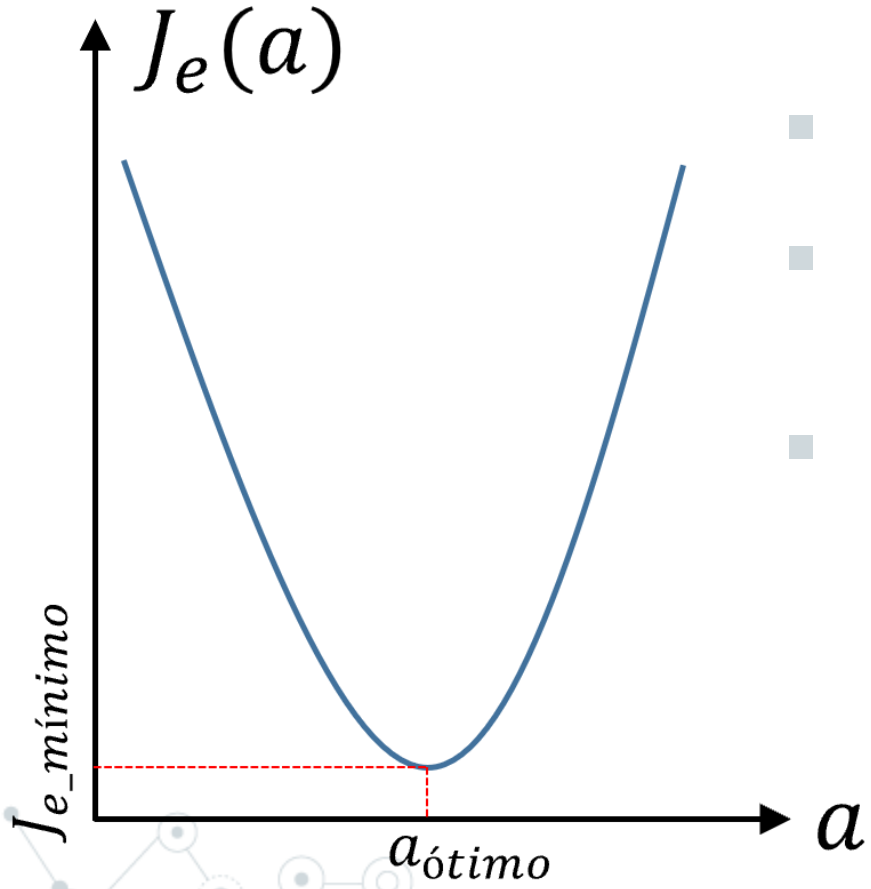
Função hipótese com um atributo.

- Antes de discutirmos como o gradiente descendente funciona, precisamos entender o que é a **superfície de erro**.
- Ela é uma representação geométrica de quão "errado" o nosso modelo está **para cada combinação possível de pesos**

$$J_e(\boldsymbol{a}) = \frac{1}{N}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{a}\|^2$$

- A plotamos variando os pesos e anotando os respectivos erros.

Superfície de Erro

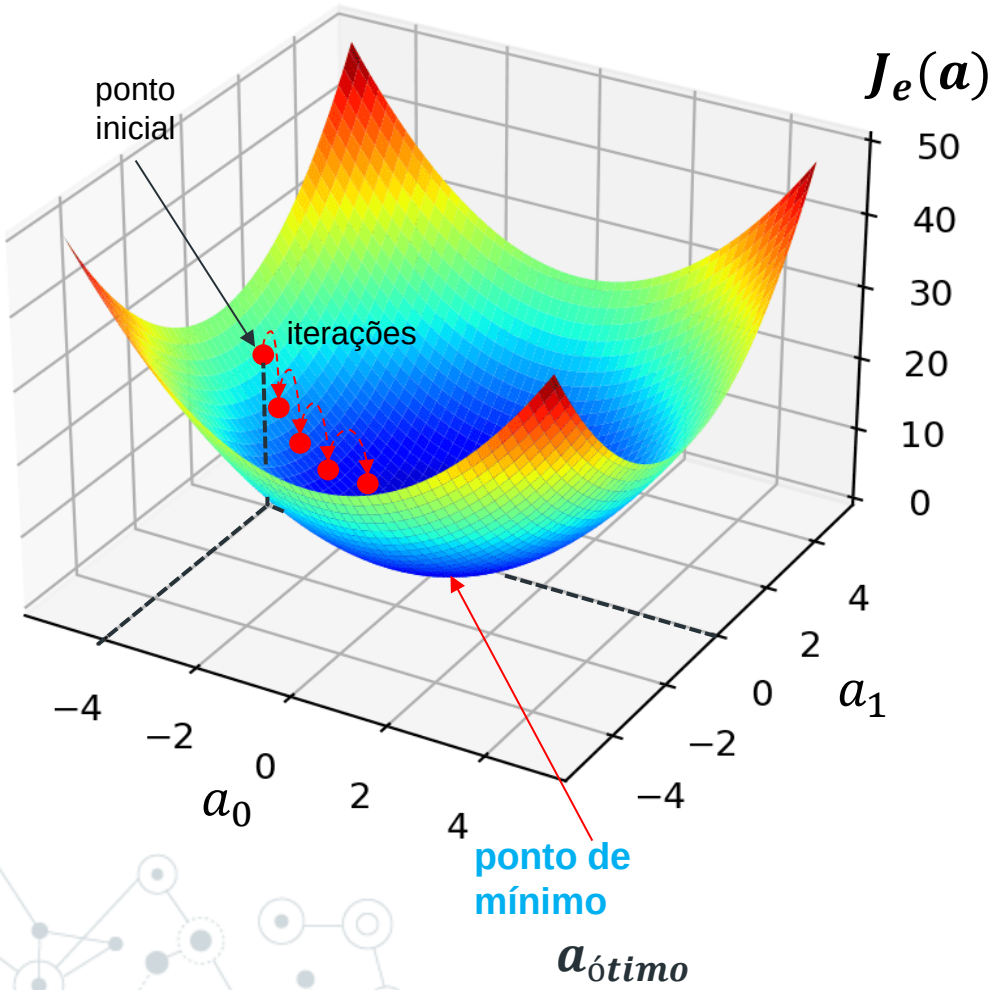


Função hipótese com um atributo.

- O ponto mais baixo nos dá os valores dos **pesos que minimizam a função de erro**.
- Quando usamos a função do EQM, por ela ter uma **forma quadrática**, a superfície resultante **terá sempre o formato de uma tigela,** ou seja, ela tem o formato de uma **parábola**.
- Além disso, devido ao termo quadrático ser positivo, ela terá a **concavidade sempre voltada para cima.**

O Que Isso Significa na Prática?

Superfície de Erro

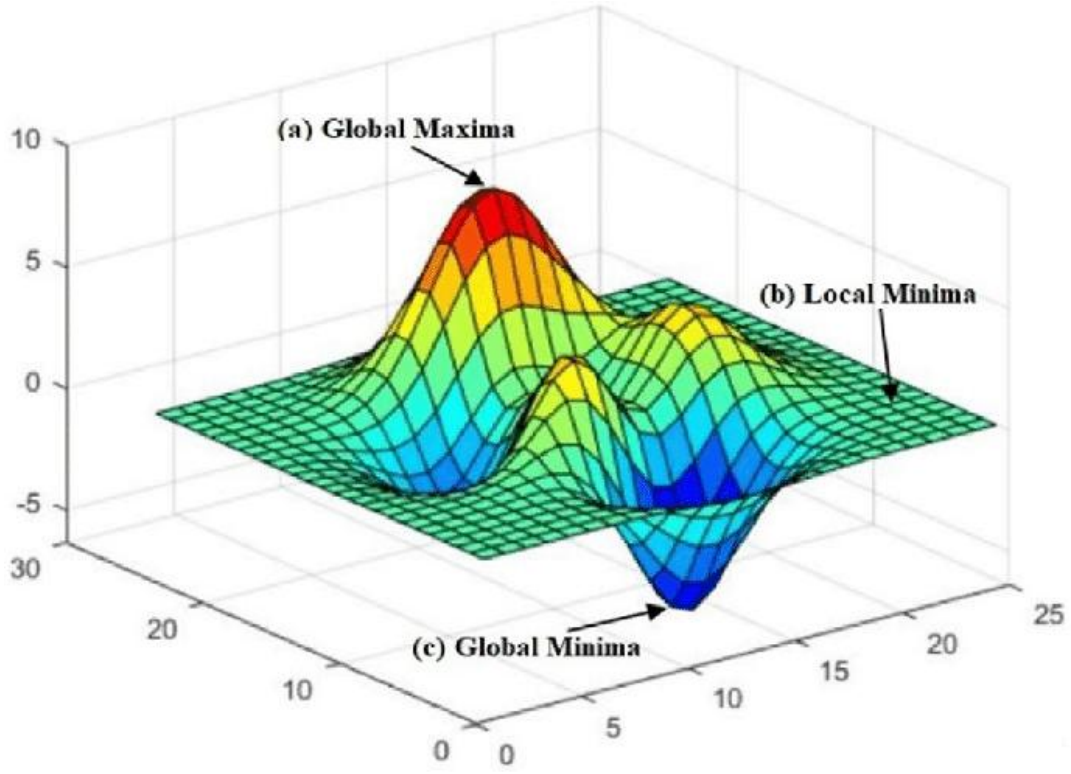


Função hipótese com dois atributos.

→ Existe apenas **um único ponto mais baixo** (o mínimo global).

→ Portanto, não importa onde valores iniciais são inicializados, o gradiente descendente **sempre** chegará à melhor solução possível.

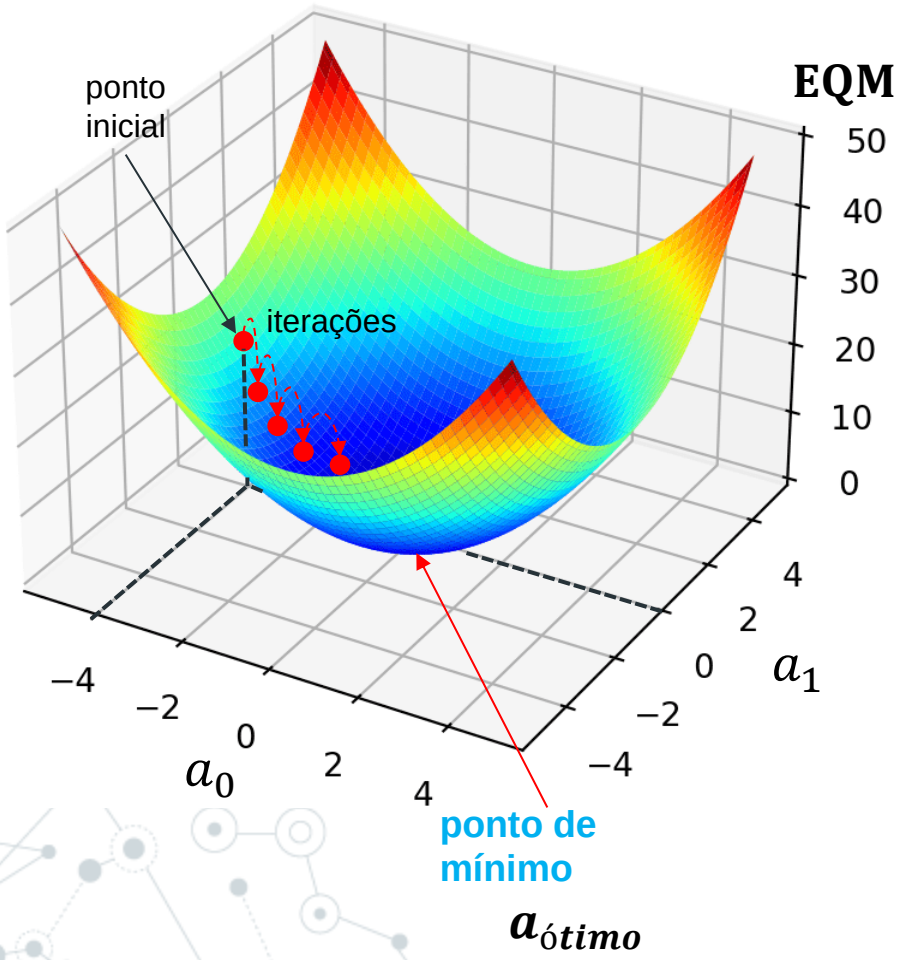
Superfície de Erro



- **Modelos não lineares**, como redes neurais, apresentam superfícies de erro totalmente irregulares, com
  - **Pontos de sela:** a função diminui em algumas direções e aumenta em outras.
  - Vários mínimos locais.
  - Platôs: região plana com erro alto.

- Essas irregularidades causam
  - treinamento mais lento
  - instabilidade numérica
  - soluções diferentes para o mesmo problema

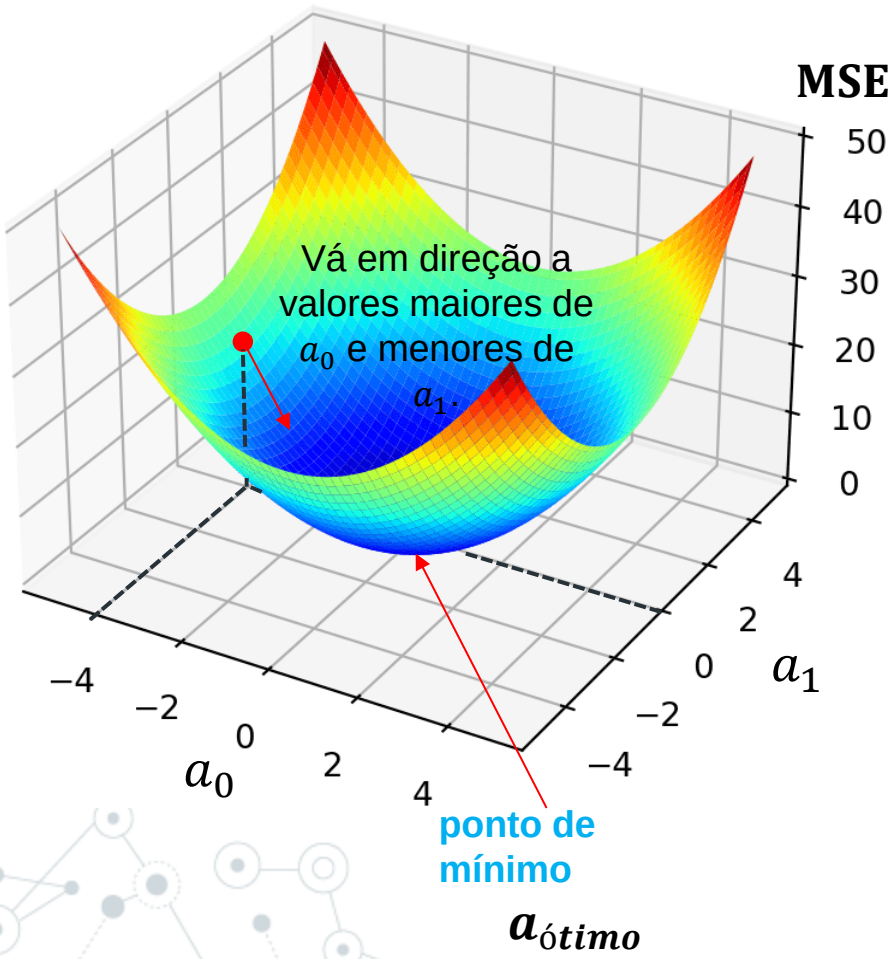
Gradiente Descendente



- Procura de forma **iterativa o ponto mais baixo da superfície de erro**, i.e., o ponto que nos dá os pesos ótimos.
- É inicializado com **pesos (i.e., em pontos) aleatórios**.
- E, a cada iteração, ele **refina** os valores dos pesos, ficando mais e mais próximos do ponto de mínimo.
- Esse ***processo de busca*** que tem como ***objetivo*** encontrar o ***ponto*** que ***minimiza o erro*** é chamado de ***otimização*** e, no contexto de ML, é conhecido como ***treinamento do modelo***.

→ Treinamento é o processo de ***atualizar o modelo*** para obter predições melhores a cada ***iteração***.

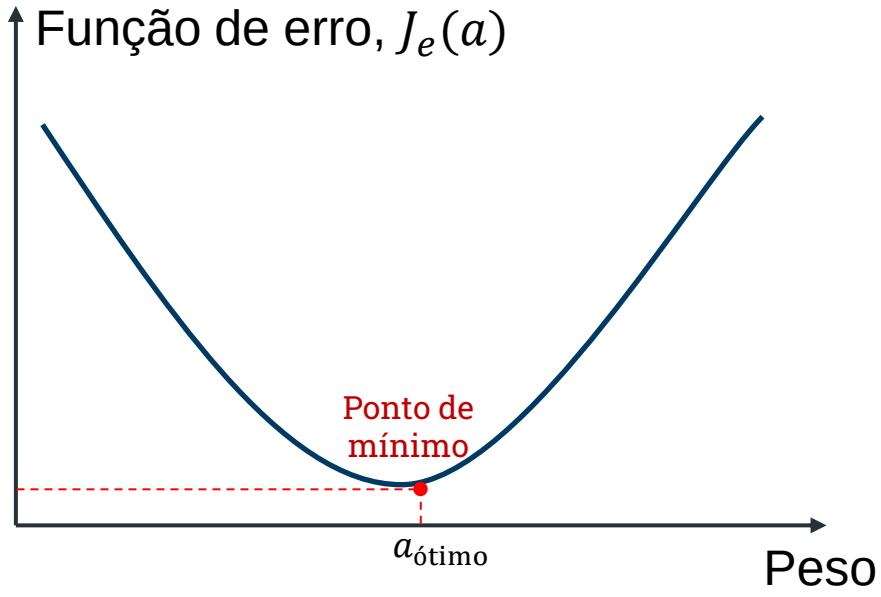
Gradiente Descendente



- O **treinamento** (ou atualização) do modelo se **baseia** na **informação obtida a partir da função de erro**.
- Essa informação **aponta para a direção do ponto de mínimo** **(i.e., conjunto de pesos ótimo)**.
- Como veremos, de posse da informação trazida pelo erro, o **gradiente descendente caminha iterativamente** através da **superfície de erro** até o **ponto de mínimo.**

Gradiente Descendente

Vetores Gradiente e Formato da Função de Erro



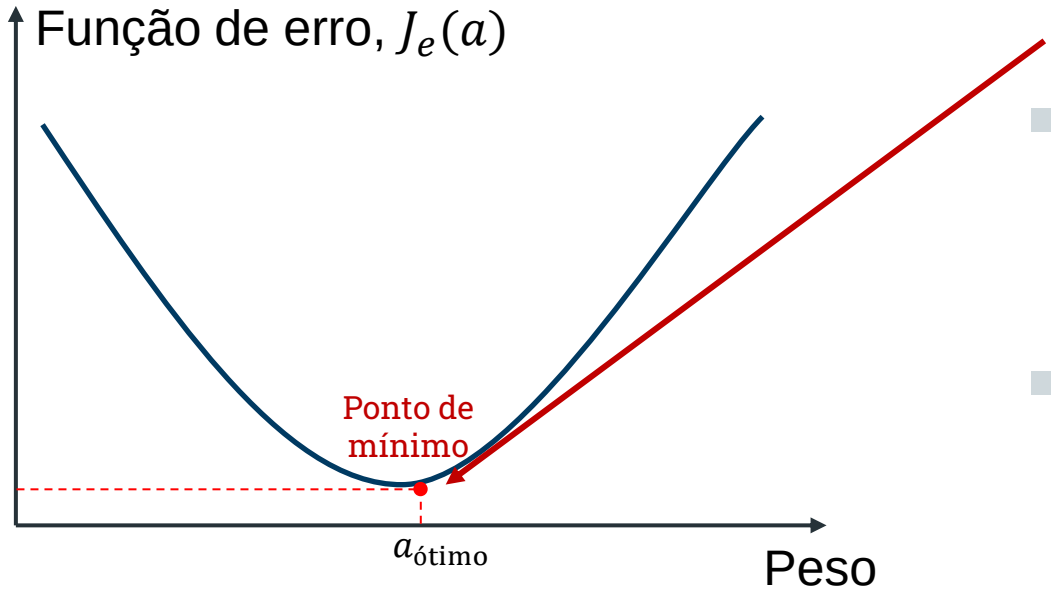
A função de erro, $J_e(a)$, quantifica a diferença entre y e $\hat{y}$.

$$J_e(a) = \text{MSE} = \frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n(a) - y_n)^2$$

- Vamos primeiro ver como os vetores gradiente nos ajudam a minimizar o erro, ou seja, como encontrar o **ponto de mínimo da função de erro**.
- A **convexidade** da superfície de erro é **importante** pois **garante** que a **função tenha apenas um ponto de mínimo, o mínimo global**.
- Isso permite que encontremos a solução ótima de forma mais eficiente.

Gradiente Descendente

Vetores Gradiente e Formato da Função de Erro



A função de erro, $J_e(a)$, quantifica a diferença entre y e $\hat{y}$.

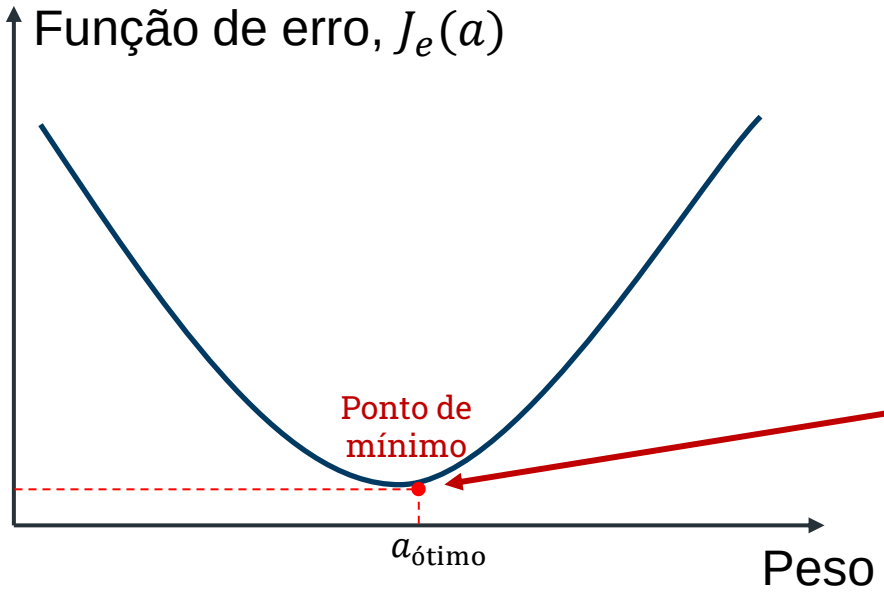
$$J_e(a) = \text{MSE} = \frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n(a) - y_n)^2$$

- Se queremos encontrar o **mínimo** da função, basta buscarmos a **parte mais baixa da parábola**.
- Não importa quais sejam os valores dos pesos e onde a função de erro é plotada no gráfico, nós **sempre teremos certeza de que o mínimo está na parte inferior da parábola**.
- Assim, se dermos um palpite (**ponto inicial**) sobre os valores dos **pesos da função hipótese**, como o mostrado ao lado, e calcularmos o **erro**, ele será **alto** e, consequentemente, saberemos que **estamos longe do ponto de mínimo**.
- **Portanto, quanto menor o erro, mais próximo estaremos do ponto ótimo.**

OBS.: Em geral, o erro nunca será igual a 0 devido aos dados estarem corrompidos por algum tipo de ruído.

Gradiente Descendente

Vetores Gradiente e Formato da Função de Erro



- **Portanto, quanto menor o erro, mais próximo estaremos do ponto ótimo.**

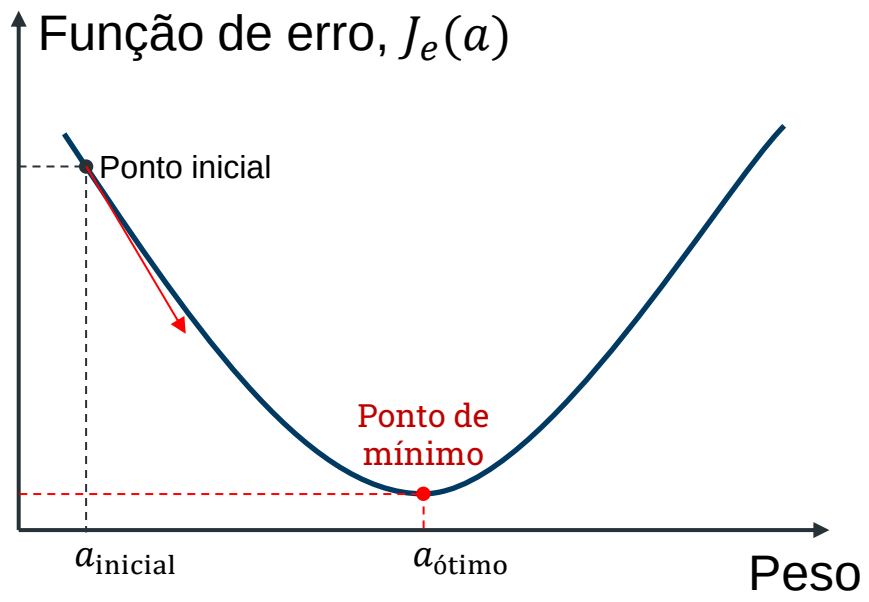
Como encontramos o ponto de mínimo através do erro?

A função de erro, $J_e(a)$, quantifica a diferença entre y e $\hat{y}$.

$$J_e(a) = \text{MSE} = \frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n(a) - y_n)^2$$

Gradiente Descendente

Vetores Gradiente



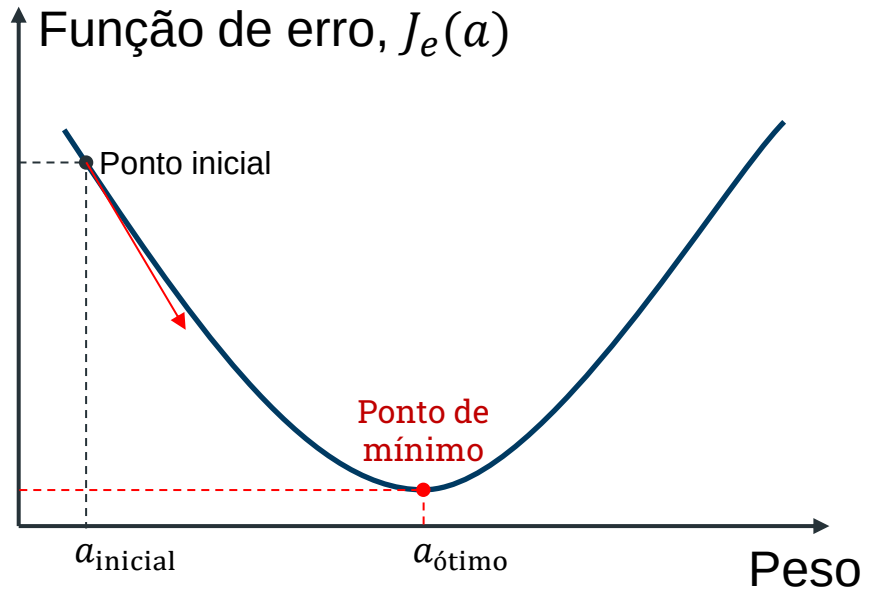
- Se nós **diferenciarmos a função de erro em um ponto qualquer** em relação aos pesos, nós obtemos o **vetor gradiente**

$$\nabla \mathrm{J_e}(\mathbf{a}) = \frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}}.$$

- Ele sempre aponta na **direção de maior taxa de crescimento da função** a partir do ponto onde ele é calculado.

Gradiente Descendente

Vetores Gradiente



No caso da função ter apenas um argumento, o vetor gradiente dá a inclinação de uma reta tangente ao ponto onde ele é calculado.

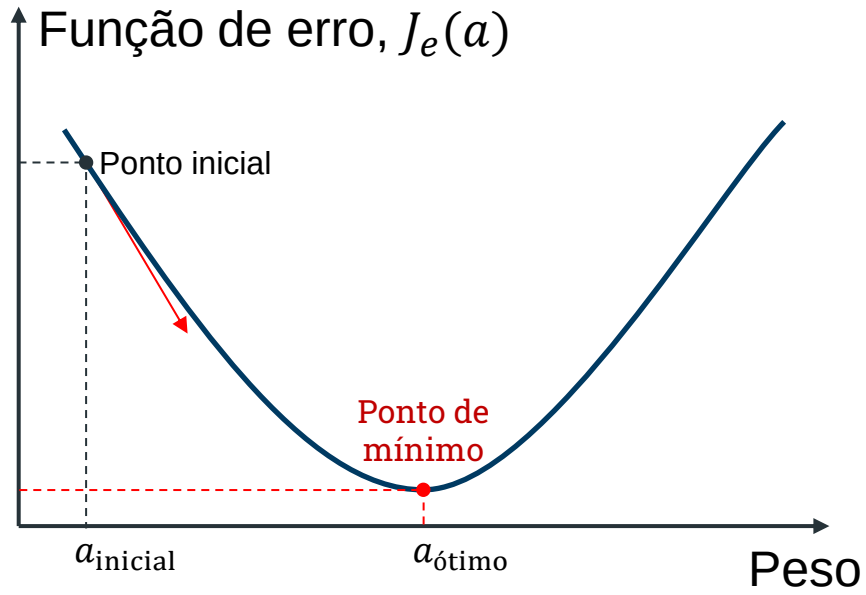
- O gradiente pode ser também interpretado como a **inclinação** de um **hiperplano tangente à função** no ponto onde ele é calculado.
  - Quanto **maior o valor absoluto** do gradiente, **mais inclinado é o hiperplano tangente** àquele ponto.
  - Portanto, **um valor igual a 0 indica inclinação nula**.

→ Onde isso ocorre?

Em pontos de MÁXIMO e de MÍNIMO.

Gradiente Descendente

Vetores Gradiente



O objetivo é minimizar a função de erro ***indo na direção contrária*** à indicada pelo gradiente.

- Porém, queremos **encontrar o mínimo** da função, o que devemos fazer?

$$\nabla \mathrm{J_e}(\mathbf{a}) = \frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}}.$$

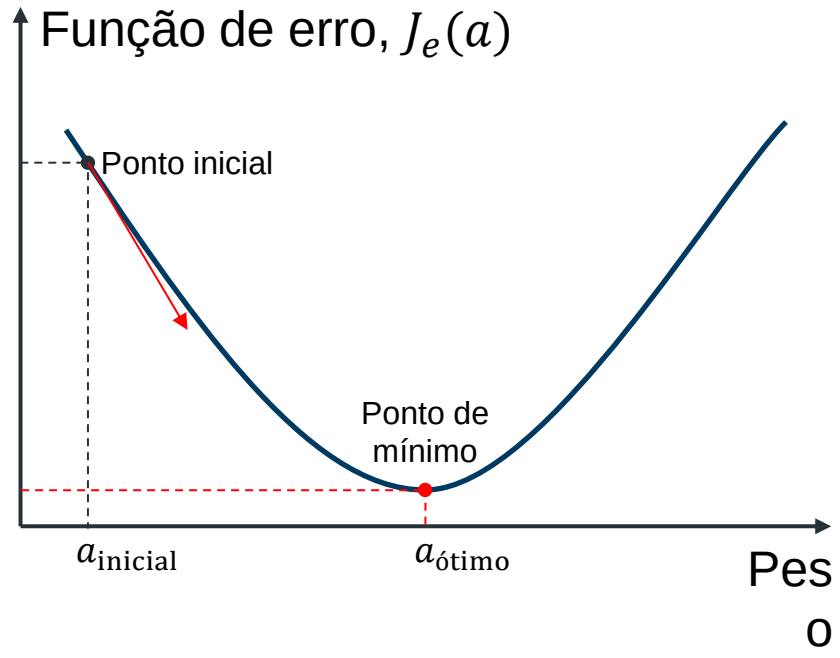
→ Basta irmos na **direção oposta** à indicada pelo vetor gradiente (i.e., negativo do gradiente)

$$-\nabla \mathrm{J_e}(\mathbf{a}) = -\frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}}$$

→ O **negativo do vetor gradiente aponta para a direção de maior taxa de decrescimento da função** a partir do ponto onde ele é calculado.

Gradiente Descendente

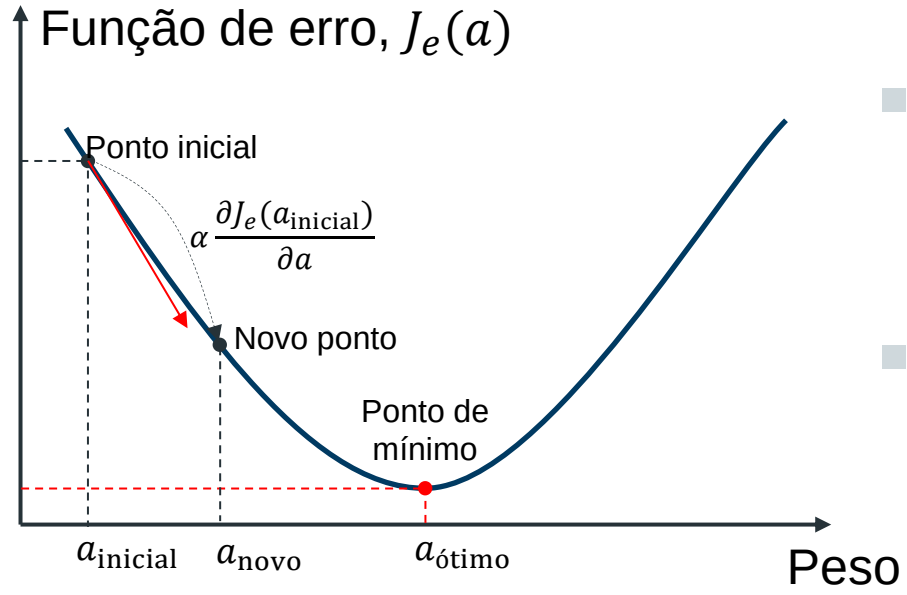
Qual a Distância até o Mínimo?



- Entretanto, o gradiente **não dá informação sobre a distância até o ponto de mínimo**, mas pelo menos sabemos a direção correta.
- Podemos fazer a analogia com uma bola solta em uma ladeira.
- A gravidade dá a direção até a parte mais baixa da ladeira, mas não dá a distância até lá.

Gradiente Descendente

Passo de Aprendizagem



- Portanto, se quisermos **seguir até o ponto de mínimo** a partir de um ponto qualquer, podemos **dar um passo na direção apontada pelo gradiente**.
- Nós sabemos a direção do ponto de mínimo e podemos **escolher o tamanho do passo** para darmos naquela direção.

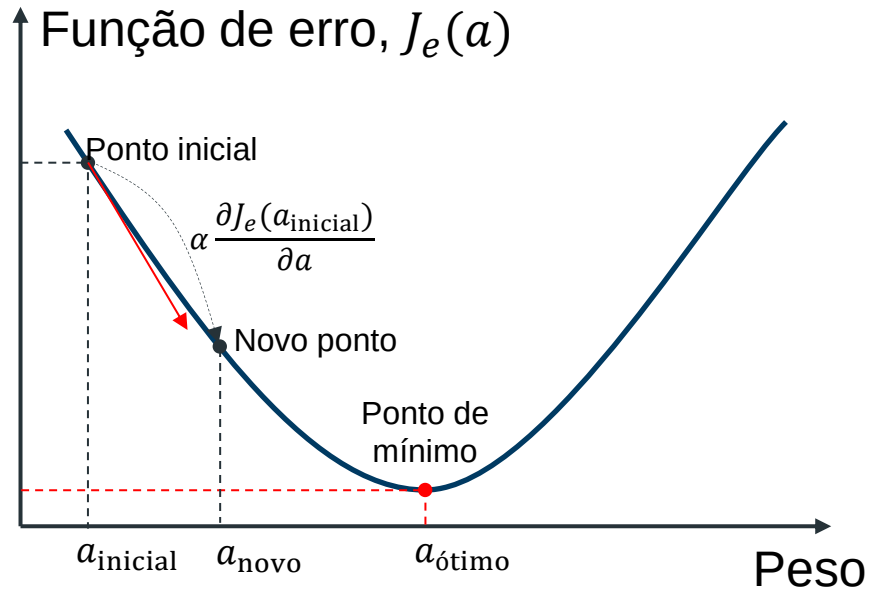
Termo de atualização

$$a_{\text{novo}} = a_{\text{inicial}} - \alpha \frac{\partial J_e(a_{\text{inicial}})}{\partial a}$$

Sentido aposto ao apontado pelo gradiente

Gradiente Descendente

Passo de Aprendizagem



- O passo de aprendizagem determina a **porcentagem da informação do vetor gradiente que é adicionada aos pesos**.
- A equação abaixo é chamada de **equação de atualização dos pesos**

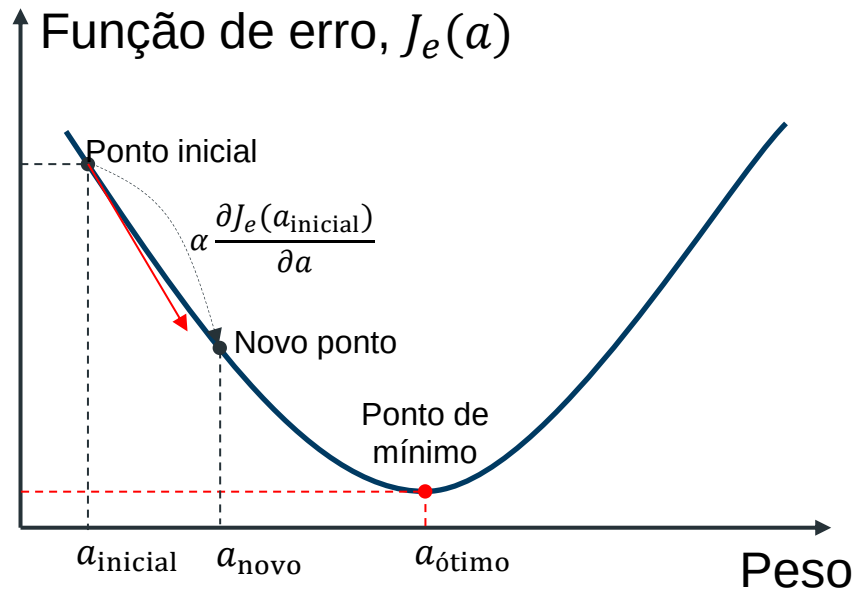
Termo de atualização

$$a_{\text{novo}} = a_{\text{inicial}} - \alpha \frac{\partial J_e(a_{inicial})}{\partial a}$$

Sentido aposto ao apontado pelo gradiente

Gradiente Descendente

Passo de Aprendizagem



- A porcentagem do gradiente é frequentemente chamada de **taxa ou passo de aprendizagem** e é, normalmente, denotado pela letra grega, α.
- Se o peso atual está à esquerda do mínimo, o **termo de atualização** faz com que o novo peso seja maior do que o anterior.
- Se o peso atual está à direita do mínimo, o **termo de atualização** faz com que o novo peso seja menor do que o anterior.

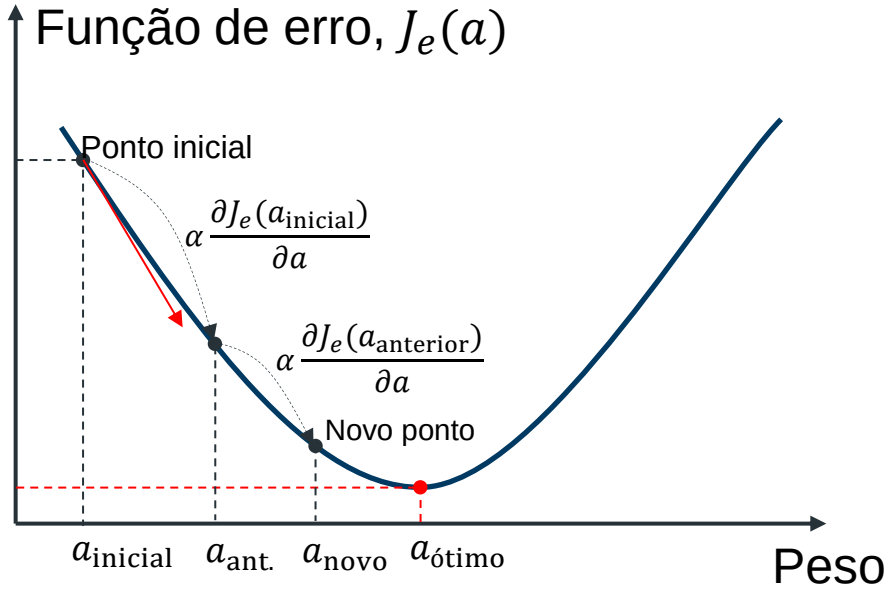
Termo de atualização

$$a_{\text{novo}} = a_{\text{inicial}} - \alpha \frac{\partial J_e(a_{\text{inicial}})}{\partial a}$$

Sentido aposto ao apontado pelo gradiente

Gradiente Descendente

Otimização Iterativa



$$a_{\text{novo}} = a_{\text{anterior}} - \alpha \frac{\partial J_e(a_{\text{anterior}})}{\partial a}$$

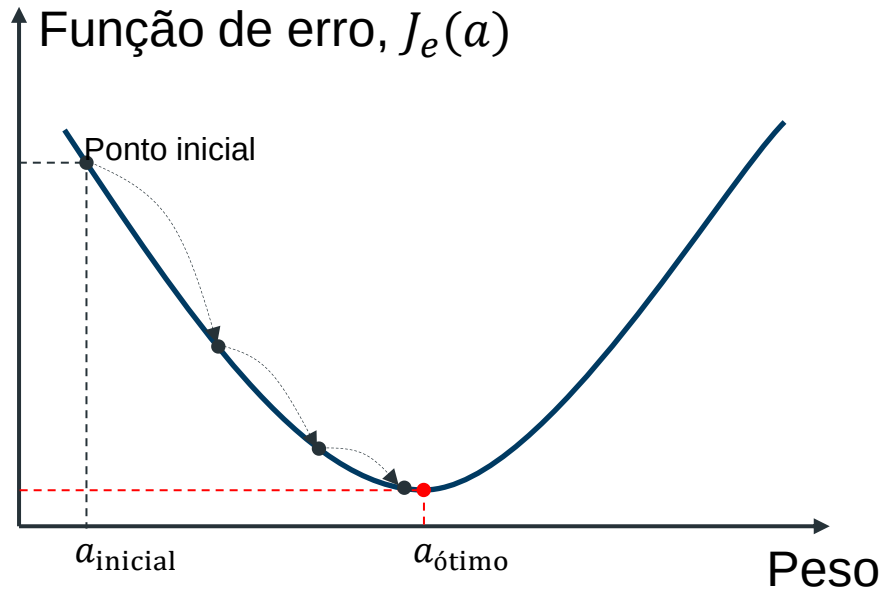
Equação de atualização dos pesos

- Portanto, dada a **direção do gradiente e um passo de aprendizagem**, agora podemos **iterativamente** caminhar em direção ao ponto de mínimo.
- A cada **iteração** calculamos o gradiente no ponto atual, atualizamos os pesos com uma porcentagem do gradiente e calculamos o gradiente no novo ponto.
- **Repetimos** esse processo **até** que a **inclinação da reta tangente ao ponto atual se torne igual a 0**, indicando que o ponto de mínimo foi atingido.

→ O que ocorre quando o gradiente é 0?

Gradiente Descendente

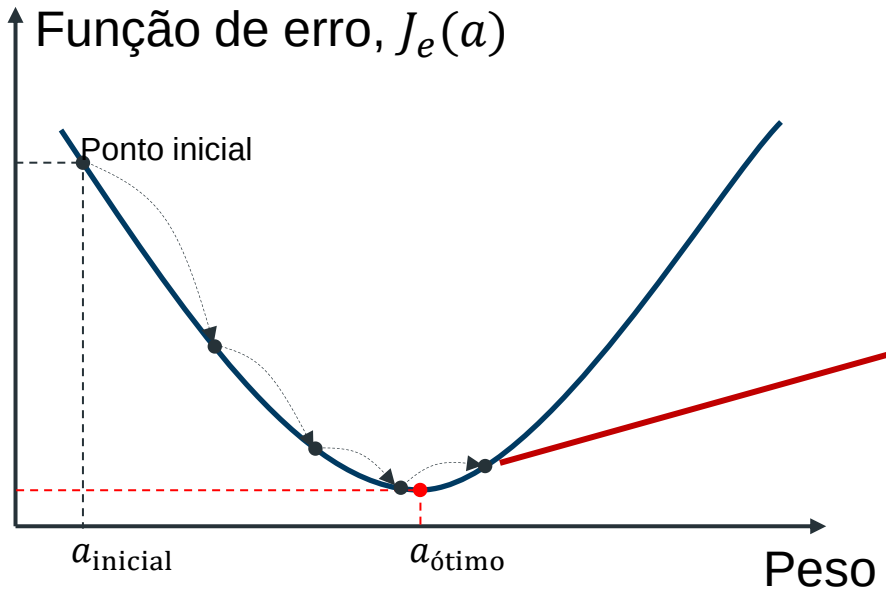
Tamanho do Passo de Aprendizagem



- O objetivo é que a cada nova iteração, nós nos movamos para mais e mais perto do ponto de mínimo, $a_{\text{ótimo}}$.
- Porém, devemos tomar **cuidado**, com o **tamanho do passo de aprendizagem**.
- O valor do passo de aprendizagem é um **hiperparâmetro** crucial para o GD.
  - **Hiperparâmetros** são parâmetros do modelo que **não são aprendidos durante o treinamento**, mas sim definidos pelo desenvolvedor antes do treinamento.
- **O passo de aprendizagem influencia a velocidade e a convergência do treinamento.**

Gradiente Descendente

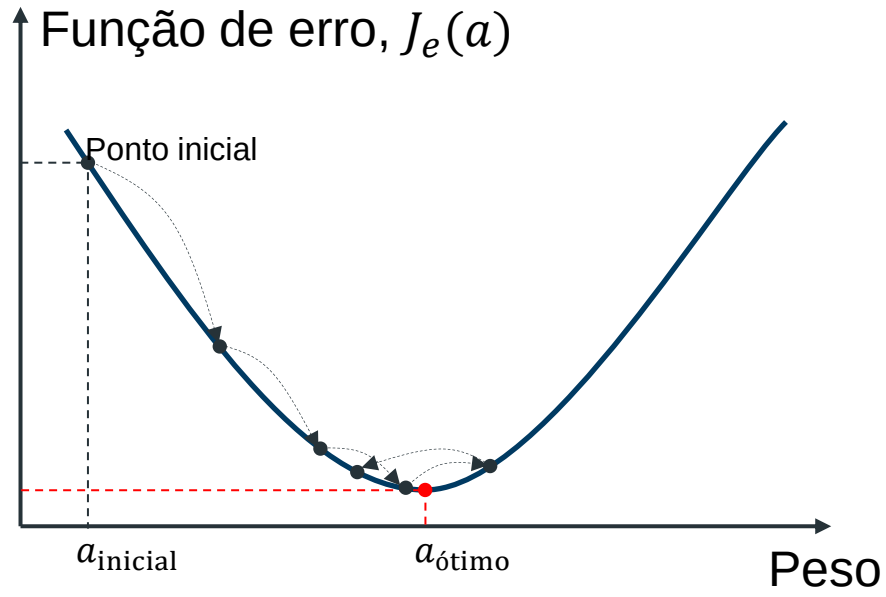
Tamanho do Passo de Aprendizagem



Se o passo de aprendizagem for **muito grande**, podemos ultrapassar o ponto de mínimo.

Gradiente Descendente

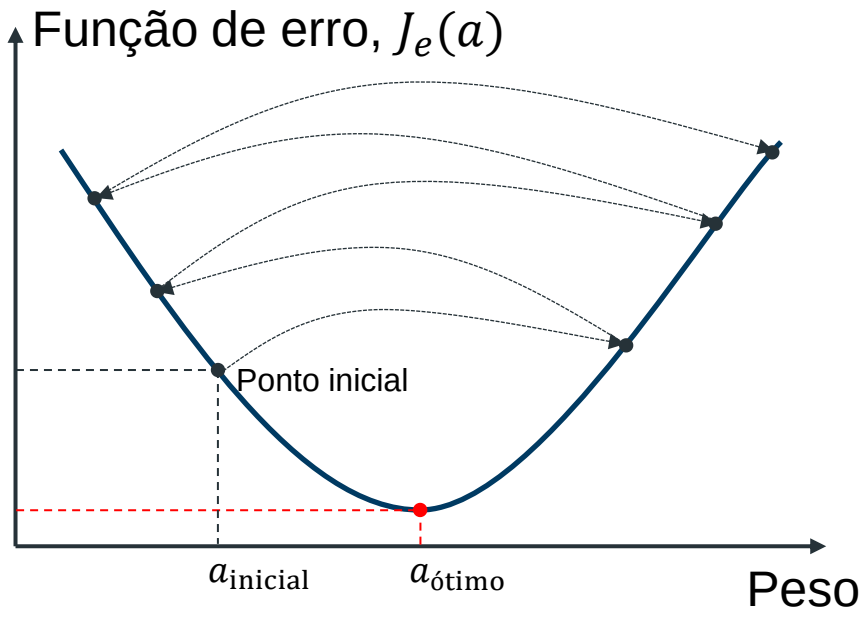
Tamanho do Passo de Aprendizagem



Se o passo de aprendizagem for **muito grande**, o processo de otimização pode ficar **ziguezagueando** de um lado para o outro do fundo da função **sem nunca atingir o ponto de mínimo**.

Gradiente Descendente

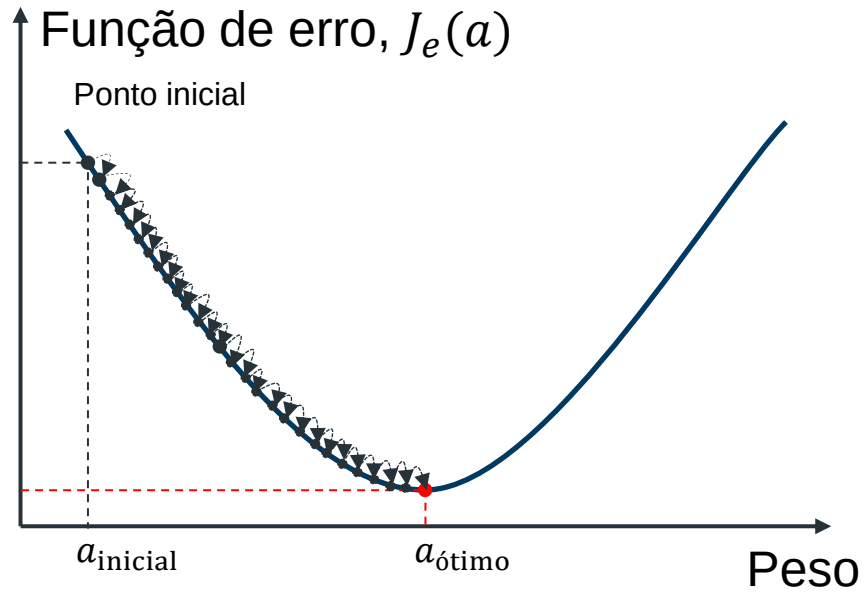
Tamanho do Passo de Aprendizagem



- Dependendo do quão grande for o valor do passo de aprendizagem, pode ocorrer até a **divergência** ao invés da convergência.
- Ou seja, ao invés de se aproximar do ponto de mínimo, o algoritmo **se distancia** dele a cada iteração.
- Se isso ocorrer, após algumas iterações, ocorre o estouro da precisão numérica das variáveis envolvidas na regra de atualização dos pesos.

Gradiente Descendente

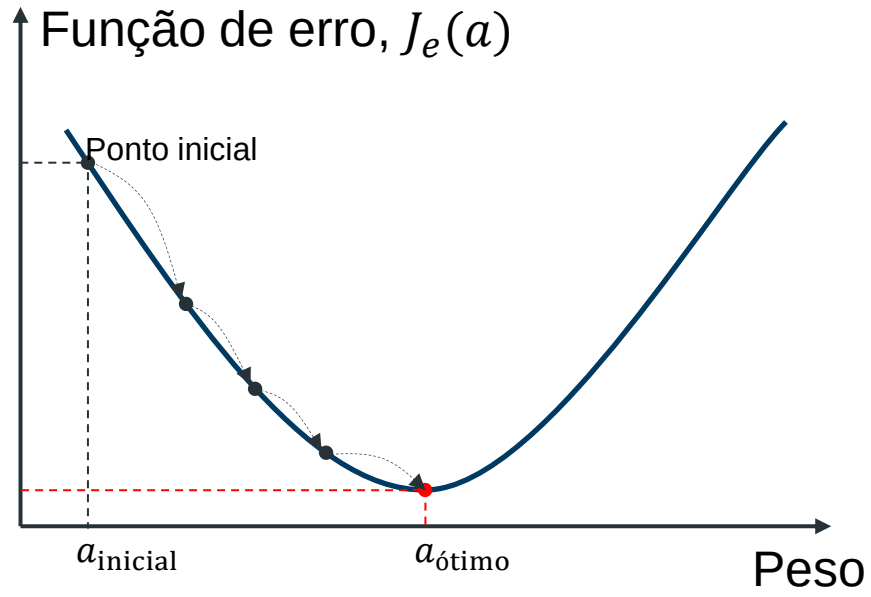
Tamanho do Passo de Aprendizagem



- Outra questão, **menos problemática** que passos grandes, é a situação oposta.
- Passos **muito pequenos** fazem com que se **leve muito tempo**, i.e., iterações, para atingir o ponto de mínimo.
- Se cada iteração levar um tempo razoável para ser executada, o tempo necessário para se atingir o ponto de mínimo pode ser muto grande.
- Porém, **se esperarmos** tempo suficiente, a **convergência é garantida**.

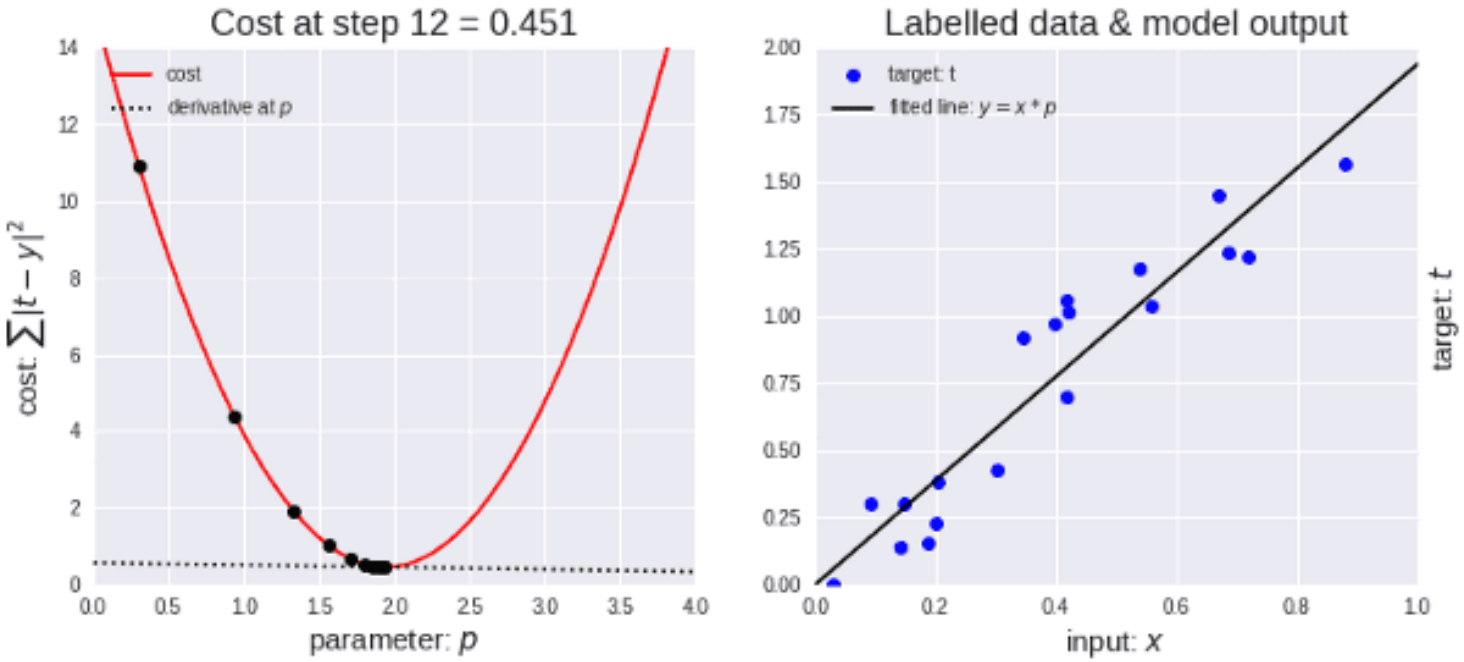
Gradiente Descendente

Tamanho do Passo de Aprendizagem



- É importante escolhermos um passo de aprendizagem que **acelere a convergência sem causar oscilações em torno do ponto de mínimo**.
- Em geral, um bom valor para o passo é encontrado por **tentativa e erro**.
- **Uma forma mais avançada é ajustar o passo ao longo das iterações.**
  - Usamos **valores grandes no início**, em pontos distantes do mínimo, e o **reduzimos gradualmente** ao longo das iterações.

Gradiente Descendente



→ A animação ao lado mostra o gradiente descendente atualizando os pesos de um modelo, no caso uma reta, para que ele se ajuste da melhor forma possível aos dados de treinamento (pontos azuis).

$\boldsymbol{a}$ ← inicializa em um ponto qualquer do espaço de pesos
loop até convergir **ou** atingir o número máximo de iterações **do**

$$\boldsymbol{a} \leftarrow \boldsymbol{a} - \alpha \frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}}$$ (***eq. de atualização dos pesos***)

Gradiente Descendente

Versões do Gradiente Descendente

- O GD pode ser implementado de **3 formas diferentes dependendo de como calculamos o vetor gradiente**.
- Para entendermos as 3 versões, vamos primeiro encontrar o vetor gradiente, $\frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}}$, e substituí-lo na equação de atualização dos pesos.
- Vamos considerar o EQM como função de erro e um hiperplano como função hipótese

$$\underbrace{\hat{y}(n) = a_0 + a_1 x_1(n) + \cdots + a_K x_K(n)}_{\text{Equação do hiperplano}} = \sum_{i=0}^{K} a_i x_i(n) = \boldsymbol{x}(n)^T \boldsymbol{a},$$

onde $\boldsymbol{x}(n) = [1 \;\; x_1(n) \;\; \ldots \; x_K(n)] \in \mathbb{R}^{K+1\times 1}$ é um vetor com todos os atributos da n-ésima amostra de treinamento.

Gradiente Descendente

Versões do Gradiente Descendente

- O **vetor gradiente** da **função de erro em relação aos pesos** é dado por

$$\frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}} = \frac{\partial}{\partial \boldsymbol{a}}\left[\frac{1}{N}\sum_{n=0}^{N-1}\left(y(n) - \hat{y}(n)\right)^2\right] = -\frac{2}{N}\sum_{n=0}^{N-1}[y(n) - \hat{y}(n)]\boldsymbol{x}(n) = -\frac{2}{N}\boldsymbol{X}^T(\boldsymbol{y} - \hat{\boldsymbol{y}}).$$

- **OBS.:** Notem que o gradiente é a média dos gradientes de todos os exemplos de treinamento.

- Usamos um hiperplano, mas todos os resultados e conclusões que tiraremos podem ser diretamente estendidos a polinômios.

Gradiente Descendente

Versões do Gradiente Descendente

- Substituindo o **vetor gradiente** na **equação de atualização dos pesos**, temos

$$\boldsymbol{a} = \boldsymbol{a} - \alpha \frac{\partial J_e(\boldsymbol{a})}{\partial \boldsymbol{a}} = \boldsymbol{a} + \alpha \frac{2}{N} \sum_{n=0}^{M-1} [y(n) - \hat{y}(n)] \boldsymbol{x}(n) .$$

- Podemos ter **3 versões diferentes, dependendo da quantidade de amostras**, M, considerados na média acima:
  - **Gradiente descendente em batelada (GDB).**
  - **Gradiente descendente estocástico (GDE).**
  - **Gradiente descendente em mini-lotes (GDML).**

Gradiente Descendente

Gradiente Descendente em Batelada

$$\boldsymbol{a} = \boldsymbol{a} + \alpha \frac{2}{N} \sum_{n=0}^{N-1} [y(n) - \hat{y}(n)] \boldsymbol{x}(n) .$$

- Calcula o gradiente usando **todas as amostras** de treinamento (M = N), resultando em uma **descida suave e direta** em direção ao mínimo.
- **Convergência** para o **mínimo global** é **garantida** quando a função de erro é **convexa e o passo não é grande demais**.
- Pode ser **computacionalmente complexo** dependendo do tamanho do modelo e do conjunto de dados.
  - **Por processar todos os exemplos, é lento em conjuntos muito grandes e consome muita memória.**
- Das 3, é a versão que **obtém os melhores resultados**.

Gradiente Descendente

Gradiente Descendente Estocástico

$$\mathbf{a} = \mathbf{a} + \alpha 2[y(n_{random}) - \hat{y}(n_{random})]\mathbf{x}(n_{random}).$$

- Utiliza **apenas uma amostra** do conjunto de treinamento (i.e., $M = 1$) para calcular uma **estimativa estocástica do gradiente**.
  - **Versão estocástica, pois a cada iteração toma-se uma amostra aleatória do conjunto para calcular a estimativa do gradiente.**
- Quando os **dados de treinamento são ruidosos**, a **estimativa** do gradiente **é ruidosa**, fazendo com que a **convergência não ocorra** ou **não seja garantida**.
- Atualiza os pesos a cada exemplo processado, gerando uma **trajetória oscilatória** que flutua ao redor do mínimo devido à estimativa ruidosa do gradiente.
- **Entretanto, é mais rápido e usa menos CPU e memória do que o GDB.**

Gradiente Descendente

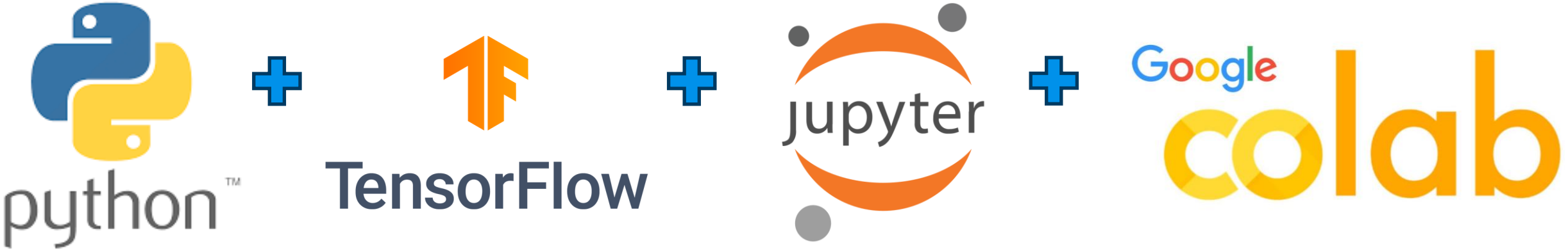
Gradiente Descendente em Mini-Lotes

$$\boldsymbol{a} = \boldsymbol{a} + \alpha \frac{2}{MB} \sum_{n=0}^{MB-1} [y(n) - \hat{y}(n)]\boldsymbol{x}(n)\,.$$

- Utiliza um ***subconjunto de amostras***, MB, do conjunto de treinamento ($M = MB$) para o cálculo do gradiente.
- Por, em geral, usar um ***subconjunto*** de exemplos, $1 < MB < N$, é mais rápido que o GDB e mais preciso e estável do que o GDE.
- Porém, por MB ser variável, essa versão é vista como uma ***generalização*** das duas versões anteriores, pois MB pode ser feito igual a 1 ou N.
- Portanto, por poder ter sua complexidade configurada, é a ***versão mais usada no treinamento de redes neurais***.

Exemplo

- Gradiente descendente.



Atividades

- Exercício: Gradiente descendente.



Obrigada pela Atenção!

Dúvidas?

victoria.souto@Inatel.br